Fine-grained Poisoning Attacks to Local Differential Privacy Protocols for Marginal Release

Xuebin Ren, Hantang Qin, Xinyu Li, Shusen Yang, Cong Zhao, Qing Han
Xi'an Jiaotong University, Xi'an, China
xuebinren@mail.xjtu.edu.cn, qinhantang@stu.xjtu.edu.cn, 2213311049@stu.xjtu.edu.cn,
shusenyang@mail.xjtu.edu.cn, congzhao@xjtu.edu.cn, qinghan@mail.xjtu.edu.cn

***Abstract*—Local Differential Privacy (LDP) enables sensitive data collection without relying on a trusted aggregator. Aiming to support various downstream analytic tasks rather than specific statistics, LDP marginal release has been widely studied to securely publish marginal distributions on multi-dimensional datasets. However, recent studies indicate that LDP protocols are highly vulnerable to data poisoning attacks. In particular, fine-grained poisoning attacks can manipulate the LDP estimates towards arbitrarily predefined target values, posing a severe security threat. Nevertheless, existing fine-grained attacks exclusively focus on other statistics, leaving the vulnerability of LDP marginal release unexplored. In this paper, we conduct a systematic study on fine-grained data poisoning attacks against LDP marginal release. Specifically, we propose a bi-level optimization-based attack framework, wherein the aggregator's reconstruction workflow serves as the lower-level problem and the attacker's goal as the upper-level one. Leveraging implicit differentiation to solve this nested optimization, our attack enables precise manipulation of LDP-estimated marginal distributions. Finally, we explore a defense mechanism to mitigate such attack. Besides theoretical analysis, we have validated both our attack and defense through extensive experiments on real-world datasets.**

I. Introduction

Local Differential Privacy (LDP) [1], [2], [3], [4], [5], [6], [7] has emerged as the *de facto* standard for privacy preservation, enabling massive data collection without a trusted aggregator. While widely studied for tasks like frequency estimation [8], [5], [9], [10], [6], [7], range queries [11], [12], [13], [14], [15] and graph analysis [16], [17], [18], [19], [20], [21], LDP has also been adapted for marginal release. Unlike other tasks, publishing marginal distributions inherently supports various downstream tasks (e.g., data synthesis), making it a crucial and extensively studied topic in LDP. Existing methodologies include the *full contingency* (FC) [10], *all marginals* (AM) [22], [23], *fourier transformation* (FT) [24] and *expectation maximization* (EM) [25] methods. Recently, the *consistent adaptive local marginal* (CALM) protocol [26] has emerged as the state-of-the-art (SOTA) solution, achieving superior utility for high-dimensional marginal release.

Despite the promising prospects of LDP, recent studies indicate that LDP protocols are vulnerable to data poisoning attacks, where an attacker can manipulate LDP-estimated results by injecting fake users to report crafted data. LDP protocols are susceptible to these attacks because data collection in LDP is performed in a distributed manner, allowing fake users to send arbitrary LDP-compliant data to the aggregator. The local perturbation nature of LDP protocols also makes these attacks highly concealed. According to the evolution of manipulation granularity, existing poisoning attacks against LDP protocols can be categorized into *untargeted*, *targeted*, and *fine-grained* attacks. Untargeted attacks [27], [28], [29], [30] blindly maximize the overall estimation error. Targeted attacks [31], [32], [33], [34], [35] improve the manipulation granularity by allowing the attacker to specify certain items, focusing on extremely promoting or demoting their estimates. Furthermore, fine-grained poisoning [36], [37] enables the attacker to manipulate the estimated results towards arbitrarily predefined target values rather than mere maximization or minimization. Despite this advanced capability, existing fine-grained attacks focus on other statistics (e.g., mean/variance [36] or streaming data [37]). The vulnerability of LDP marginal release protocols, especially under fine-grained manipulations, remains unexplored.

In this work, we conduct a systematic study on fine-grained data poisoning attacks against LDP marginal release protocols, where the attacker aims to manipulate the aggregator's final estimated marginal table (i.e., marginal distribution) towards an attacker-predefined target marginal table. We primarily focus on attacking the SOTA CALM protocol. Note that our attack can be naturally applied to other LDP marginal release protocols (see Sec. VI). Such fine-grained attacks can cause devastating consequences in practice. For instance, when a medical institution collects a multi-dimensional dataset (containing the attributes of age, occupation, and disease status) under LDP, the attacker can manipulate the marginal table of occupation and disease status. By falsely inflating the ratio of infected individuals in a specific occupation (e.g., teachers), the attack could mislead policymakers into implementing unnecessary quarantines or misallocating medical resources. Achieving such an attack involves three technical challenges:

- **Multi-dimensional Distribution Manipulation.** Unlike existing attacks that target isolated single attributes, LDP marginal release involves complex correlations among multiple attributes. It is challenging for the attacker to orchestrate the fake users' reports across different attributes and systematically manipulate the entire distribution.
- **Complex Reconstruction Formulation.** LDP marginal release protocols typically employ complex reconstruction workflows to derive the final estimate. Due to the inherent non-differentiable operations within these workflows, it is challenging to mathematically model them and incorporate them into the attack model to achieve precise manipulation.
- **Implicit Attack Solving.** Even if the reconstruction work-

flow can be formulated, solving the attack model remains non-trivial. Since the aggregator's reconstruction involves iterative processes, there is no explicit analytical mapping between the final estimate and the fake users' reports. Thus, it is challenging to resolve this implicit dependency and derive the optimal fake users' reports.

To address the above challenges, we propose a novel bi-level optimization framework. Specifically, we formulate the attacker's goal of approaching the target marginal table as the upper-level problem, and embed the aggregator's reconstruction procedures as the lower-level problem. To solve this optimization problem with nested dependency, we utilize implicit differentiation to calculate the gradients and iteratively update the malicious reports. Extensive experiments on real-world datasets demonstrate the effectiveness of our attack. Finally, we propose a defense mechanism to mitigate these threats. Our main contributions are summarized as follows:

- To the best of our knowledge, we are the first to investigate the vulnerability of LDP marginal release protocols, especially under the fine-grained data poisoning. Our work highlights the security of LDP marginal release protocols.
- We propose a novel bi-level optimization framework to compromise LDP marginal release protocols. By embedding the aggregator's reconstruction workflow into the optimization and utilizing implicit differentiation, our framework enables precise manipulation of the final marginal tables.
- We implement our proposed attacks and conduct extensive experiments on real-world datasets. Results show the superior performance of our proposed attacks. Alongside the attacks, we explore and validate a defense mechanism capable of mitigating the proposed attacks.

II. Background Knowledge

A. Local Differential Privacy (LDP)

In LDP, $\mathcal{M}$ is a randomized mechanism that perturbs each user's private data v before sending it to the aggregator.

Definition 1 (Local Differential Privacy)**.** *A mechanism $\mathcal{M}$ with $\mathcal{O}$ denoting the set of all possible outputs of $\mathcal{M}$ satisfies ϵ-LDP if and only if, for $\forall\ v, v' \in Dom(\mathcal{M})$ and $\forall S \subseteq \mathcal{O}$ there is $Pr[\mathcal{M}(v) \in S] \leq e^{\epsilon} Pr[\mathcal{M}(v') \in S]$.*

B. Frequency Estimation under LDP

LDP data analytics are commonly built on Frequency Oracles (FOs), which enable frequency estimation of any item in a domain $\Omega_{FO} = \{\omega_1, \omega_2, \ldots, \omega_{d_{FO}}\}$ of size d_{FO}. Common FO protocols include GRR [1], OUE [10] and Ada [26].

GRR. A common FO is *Generalized Randomized Response* (GRR). In GRR, every user j sends their private data $v^j \in \Omega_{FO}$ to the aggregator with probability $p = \frac{e^{\epsilon}}{e^{\epsilon}+d_{FO}-1}$, or randomly sends another item in a candidate set $\Omega_{FO} \backslash \{v^j\}$ with probability $q = \frac{1}{e^{\epsilon}+d_{FO}-1}$. The aggregator estimates the frequency of each distinct item $\omega_i \in \Omega_{FO}$, i.e., $\hat{f}[i]$, as follows:

$$\hat{f}[i] = \frac{1}{p-q}\left(\frac{1}{n}\sum_{j=1}^{n} \mathbb{I}^{(i)}_{S(y^j)} - q\right), \tag{1}$$

where n is the number of users, y^j is the item sent by j-th user, and $S(y^j) = \{y^j\}$ is the support set of y^j. $\mathbb{I}^{(i)}_{S(y^j)}$ is an indicator function. It outputs 1 if $\omega_i \in S\left(y^j\right)$ and 0 otherwise.

OUE. Another FO is *Optimal Unary Encoding* (OUE). This protocol first encodes each user's private data v^j into a binary string of length d_{FO}, where the i-th bit ($i \in \{1, \ldots, d_{FO}\}$) is set to 1 if $v^j = \omega_i$ and 0 otherwise. Then, for each bit in the encoded string, if it is 1, OUE keeps it as 1 with probability $p = 0.5$ (i.e., perturbs it to 0 with probability $1-p$). If it is 0, OUE perturbs it to 1 with probability $q = \frac{1}{e^{\epsilon}+1}$ (i.e., keeps it as 0 with probability $1-q$). After that, the perturbed string y^j is sent to the aggregator. The aggregator also estimates the frequency of each item $\omega_i \in \Omega_{FO}$ as $\hat{f}[i]$ using Eq. (1), where the support set is $S(y^j) = \{\omega_i | \omega_i \in \Omega_{FO}$ and $y^j[i] = 1\}$. And $y^j[i]$ is the i-th bit of y^j.

Ada. To reduce estimation error, *Adaptive FO* (Ada) is proposed. It adaptively chooses the above two protocols based on the domain size d_{FO} and privacy budget ϵ. Specifically, if $d_{FO} < 3e^{\epsilon} + 2$, GRR is used; otherwise, OUE is used.

C. Marginal Release under LDP

System Model. We consider marginal release over massive users with LDP. We assume there are n users and one aggregator. Let $\mathcal{A} = \{a_1, a_2, \ldots, a_d\}$ be the set of d attributes. Each attribute a_i has s_i possible values, and its domain is denoted as $V_{a_i} = \{\omega_1^i, \omega_2^i, \ldots, \omega_{s_i}^i\}$. The private data of the j-th user is represented as a d-dimensional vector $v^j = \langle v_1^j, v_2^j, \ldots, v_d^j \rangle,\ \forall v_i^j \in V_{a_i}$. The entire domain of all user records is $D = V_{a_1} \times V_{a_2} \times \cdots \times V_{a_d}$ with the size of $|D| = \prod_{i=1}^{d} s_i$, where $\times$ is the Cartesian product. In marginal release, the aggregator is interested in the marginal distribution (i.e., marginal table) M_A over a target attribute set A which contains k attributes. Specifically, assume $A = \{a_{i_1}, a_{i_2}, \ldots, a_{i_k}\} \subseteq \mathcal{A}$, where $1 \leq i_1 < i_2 < \cdots < i_k \leq d$. Denote the value space of A as $V_A = \prod_{r=1}^{k} V_{a_{i_r}} = V_{a_{i_1}} \times \cdots \times V_{a_{i_k}}$. Let $v^j[A] = \langle v_{i_1}^j, v_{i_2}^j, \ldots, v_{i_k}^j \rangle$ denote the projection of user j's data v^j onto the attribute set A. For any value combination $v \in V_A$, the genuine frequency of value v among all the users can be calculated $M_A(v) = \frac{1}{n}\sum_{j=1}^{n} \mathbb{I}[v^j[A] = v]$, where $\mathbb{I}[v^j[A] = v]$ equals 1 if $v_{i_r}^j = v_r$ for all $r \in \{1, 2, \ldots, k\}$, and 0 otherwise. v_r is the r-th entry of the combination v. By traversing all possible combinations v in V_A in lexicographical order, the entire marginal table M_A can be represented as a column vector of dimension $|V_A| = \prod_{r=1}^{k} s_{i_r}$, i.e., $M_A = (M_A(v))_{v \in V_A}$, where $M_A(v)\ \forall v \in V_A$ is non-negative and $\sum_{v \in V_A} M_A(v) = 1$. In the LDP setting, the aggregator cannot directly access the private data v^j to calculate M_A. Instead, it only receives the perturbed reports from users. The aggregator aims to calculate an estimate $T_A(v)$ for each genuine frequency $M_A(v)$ based on these perturbed reports, thereby obtaining the estimated marginal table $T_A = (T_A(v))_{v \in V_A}$. Similar to M_A, we also represent T_A as a $|V_A|$-dimensional column vector, where $v \in V_A$ is traversed in lexicographical order. Figure 1(a) shows an example of calculating marginal tables M_A and T_A when $A = \{\text{age}\}$.

LDP Marginal Release Protocols. To achieve marginal release under LDP, several protocols have been proposed, including FC [10], AM [22], [23], FT [24], and EM [25]

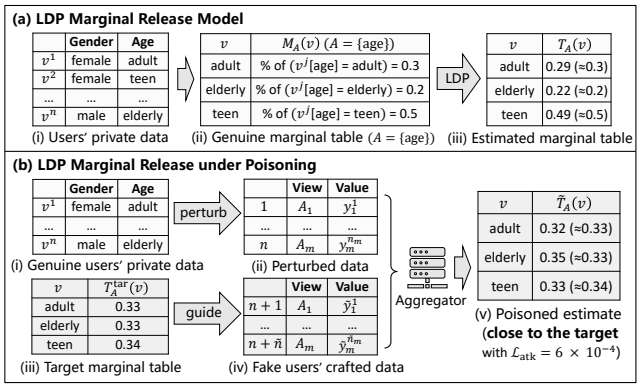


Fig. 1: Examples of LDP marginal release model and LDP marginal release under poisoning where $A = \{\text{age}\}$.

methods. Among all the protocols, CALM [26] has emerged as the SOTA solution due to its superior utility in high-dimensional data publication. Therefore, we identify it as our primary attack objective and briefly introduce it in this section. We will introduce and attack other protocols in Sec. VI.

Given the privacy budget ϵ, the total number of users n, the attribute set $\mathcal{A}$ of size d, and the target attribute set A of size k, CALM constructs an estimated marginal table T_A of the genuine one M_A. Specifically, CALM consists of the following three phases.

1) View Selection and User Reporting. The aggregator first determines the number of views (i.e., attribute sets) to be constructed m and the number of attributes contained in each view l. m and l influence the noise error (NE), sampling error (SE), and reconstruction error (RE) of CALM, where NE is calculated as $NE = \min\{\frac{4e^\epsilon}{(e^\epsilon-1)^2}, \frac{2^l-2+e^\epsilon}{(e^\epsilon-1)^2}\} \cdot \frac{2^l \cdot d}{n \cdot l}$. And the upper bound of SE is calculated as $SE = m/n$. While smaller values of l and m reduce both NE and SE, they weaken the ability of the view set to cover the target attribute set, thereby increasing RE. To balance RE, CALM introduces an error threshold θ as a constraint. It first identifies the maximum l such that $k \cdot NE < \theta$. We denote it as l_u. If $l_u \geq k$, CALM will search for a pair (m, l) that minimizes the error $\max\{k \cdot NE, SE\}$ and holds $l \in [k, l_u]$. If $l_u < k$, it will fix $l = l_u$ and selects the largest possible m such that $SE < \theta$.

After determining m and l, the aggregator constructs m views (i.e., attribute sets) $A_1, A_2, \ldots, A_m \subseteq \mathcal{A}$ with $|A_i| = l$ and partitions the users into m disjoint groups. Each user in the i-th group projects their genuine data v^j onto the corresponding attribute set A_i, and then reports this projected value using the Ada protocol with the privacy budget ϵ and domain $\Omega_{FO} = V_{A_i}$ of size $d_{FO} = |V_{A_i}|$. $V_{A_i} = \prod_{a_j \in A_i} V_{a_j}$ is the value space of A_i. Upon collecting the reports, the aggregator estimates the frequency of each possible value combination $v \in V_{A_i}$ as $T_{A_i}(v)$ via Eq. (1) and forms the initial estimated marginal table for view A_i, denoted as $T_{A_i} = (T_{A_i}(v))_{v \in V_{A_i}},\ \forall i \in \{1, ..., m\}$.

2) Non-negativity and Consistency. CALM adopts the following processes to reduce the massive LDP noise in T_{A_i}.

Non-negativity: Given a threshold $\theta_{\text{neg}} > 0$, for any $v \in V_{A_i}$ with $T_{A_i}(v) < -\theta_{\text{neg}}$, CALM first sets $T_{A_i}(v)$ to 0. To keep the sum of the frequencies in T_{A_i} unchanged, the absolute value $|T_{A_i}(v)|$ is then subtracted from its neighbors, i.e., $T_{A_i}(v')$ where $v' \in V_{A_i}$ differs from v in one attribute value. This process iterates until no frequency is smaller than $-\theta_{\text{neg}}$.

Consistency: This process coordinates conflicting estimates among overlapping views, forcing them to yield identical distributions over any shared attribute set $A_{sha} \subseteq \mathcal{A}$. To achieve this, this process traverses all subsets A_{sha} of $\mathcal{A}$ in a topological sorting order, starting from the empty set $\emptyset$. For each specific subset A_{sha}, the consistency is performed as follows. Let $\mathcal{I}_{A_{sha}} = \{i \mid A_{sha} \subseteq A_i, 1 \leq i \leq m\}$ be the set of indices of views covering A_{sha}. CALM first derives a marginal estimate of A_{sha} from each covering view A_i ($i \in \mathcal{I}_{A_{sha}}$) by marginalizing out the remaining attributes, i.e., for $\forall v \in V_{A_{sha}}$, calculating $T_{A_i \to A_{sha}}(v) = \sum_{v' \in V_{A_i}, v'[A_{sha}]=v} T_{A_i}(v')$, where $v'[A_{sha}]$ denotes the projection of the combination v' onto the attribute set A_{sha}. Because of independent noise injection, different views may yield conflicting estimates. To resolve this, CALM refines the marginal estimate of A_{sha} via weighted averaging, deriving

$$T_{A_{sha}}(v) = \frac{\sum_{i \in \mathcal{I}_{A_{sha}}} \frac{1}{C^i_{sha}} T_{A_i \to A_{sha}}(v)}{\sum_{i \in \mathcal{I}_{A_{sha}}} \frac{1}{C^i_{sha}}}, \forall v \in V_{sha}, \quad (2)$$

where $C^i_{sha} = \prod_{a_j \in A_i \setminus A_{sha}} s_j$. After obtaining $T_{A_{sha}}$, for all views A_i ($i \in \mathcal{I}_{A_{sha}}$) and all $v' \in V_{A_i}$, the aggregator updates $T_{A_i}(v')$ in view A_i to a consistent frequency $T^c_{A_i}(v')$ by

$$T^c_{A_i}(v') \leftarrow T_{A_i}(v') + \frac{1}{C^i_{sha}} \left(T_{A_{sha}}(v'[A_{sha}]) - T_{A_i \to A_{sha}}(v'[A_{sha}])\right).$$

With this process, the aggregator can update the initial estimate T_{A_i} to the consistent marginal table $T^c_{A_i} = (T^c_{A_i}(v))_{v \in V_{A_i}},\ \forall i \in \{1, ..., m\}$.

3) Marginal Table Generation. Given the target attribute set A, CALM generates the estimated marginal table $T_A = (T_A(v))_{v \in V_A}$ from the consistent marginal tables $\{T^c_{A_i}\}_{i=1}^m$. If there exists a view A_i that fully covers A (i.e., $A \subseteq A_i$), T_A can be directly obtained by marginalizing $T^c_{A_i}$. However, if no single view fully covers A, CALM generates T_A using the Maximum Entropy method. Specifically, T_A is derived by solving the following problem:

$$\begin{aligned} \max_{T_A} \quad & -\sum\nolimits_{v \in V_A} T_A(v) \log T_A(v) \\ \text{s.t. } & \forall v \in V_A,\ T_A(v) \geq 0,\ \forall A_i,\ T_A \overset{\cap}{=} T^c_{A_i}, \end{aligned} \quad (3)$$

where $\forall A_i,\ T_A \overset{\cap}{=} T^c_{A_i}$ ensures that for any view A_i, the marginal distributions of the estimated table T_A and the consistent table $T^c_{A_i}$ over their shared attributes $A_i \cap A$ align with each other, i.e., $\forall v \in V_{A_i \cap A},\ \sum_{v' \in V_A, v'[A_i \cap A]=v} T_A(v') = \sum_{v'' \in V_{A_i}, v''[A_i \cap A]=v} T^c_{A_i}(v'')$. T_A serves as the final estimate of the genuine marginal table M_A.

III. Threat Model

A. Assumptions

We assume an attacker can inject some fake users into a LDP marginal release protocol. Thus, the protocols involves an aggregator, genuine users, and fake users controlled by the attacker. The attacker can manipulate the fake users to send arbitrary LDP-compliant data (i.e., categorical item for GRR

and binary string for OUE) to the aggregator. Specifically, we assume there are n genuine users and $\tilde{n}$ fake users. Thus, the total number of users in the protocol is $N = n + \tilde{n}$. This assumption is realistic in the real world. On the one hand, an existing study has shown that a fake user costs only \$0.004-0.50 on the black market [38]. On the other hand, analyses of social media platforms have revealed that fake accounts constitute a non-negligible fraction of users, accounting for 20% on average during worldwide social events [39]. These studies provide empirical support for injecting fake users to the LDP protocol. Recall that CALM partitions users into m views $A_1, \ldots, A_m \subseteq \mathcal{A}$. We denote the number of genuine and fake users in view A_i as n_i and $\tilde{n}_i$, respectively, satisfying $\sum_{i=1}^{m} n_i = n$ and $\sum_{i=1}^{m} \tilde{n}_i = \tilde{n}$. The genuine users' reports in view A_i are denoted as $\{y_i^1, \ldots, y_i^{n_i}\}$, while the crafted reports submitted by fake users are denoted as $\{\tilde{y}_i^1, \ldots, \tilde{y}_i^{\tilde{n}_i}\}$. Following existing studies [31], [32], [36], we assume the fake users can bypass the LDP perturbations and send these crafted data directly to the aggregator.

Since LDP protocols execute locally on users' side, the attacker can have access to the implementation of LDP protocols [31], [32]. Thus, we assume that the attacker knows the parameters of the protocol, including the privacy budget ϵ, the view attribute sets $\{A_i\}_{i=1}^m$ determined by CALM, and the specific FO mechanism adopted by the Ada protocol (i.e., GRR or OUE). Furthermore, to comply with transparency principles [5], [33] and data protection regulations [40], [41], the aggregator is typically obligated to disclose the intended analytical objectives to all users prior to data collection. Thus, we assume the attacker knows the target attribute set A.

Following existing studies [36], [37], [28], we also assume the attacker can estimate the number of genuine users n as n^e from public sources because service providers often disclose this information under LDP protection for publicity [36]. Since CALM uniformly assigns users across the m views, the attacker can further estimate the number of genuine users allocated to the i-th view A_i as $n_i^e = n^e/m$. We also assume the attacker can estimate the data distribution of the genuine users. Specifically, we assume the attacker can snoop h out of n genuine users for obtaining their private data $\{v^1, v^2, \ldots, v^h\}$ [36], [37]. Based on these data, the attacker can construct an joint frequency distribution over the entire domain D. For any value combination $u \in D$, the estimated frequency can be calculated as $f^e(u) = \frac{1}{h}\sum_{j=1}^{h} \mathbb{I}[v^j = u]$, where $\mathbb{I}[v^j = u]$ outputs 1 if the snooped data v^j equals the combination u and 0 otherwise. To estimate the data distribution on each view, the attacker marginalizes the joint distribution f^e onto the corresponding attributes. Specifically, for the i-th view A_i and any value combination $v \in V_{A_i} = \prod_{a_j \in A_i} V_{a_j}$, the estimated marginal frequency on view A_i is calculated as $f_{A_i}^e(v) = \sum_{u \in D, u[A_i]=v} f^e(u)$, where $u[A_i] = v$ indicates that the projection of u onto the attribute set A_i equals v. By traversing all possible combinations in V_{A_i}, the attacker derives the estimated frequency vector $f_{A_i}^e = (f_{A_i}^e(v))_{v \in V_{A_i}}$ for view A_i.

TABLE I: Notations.

Notation	Description
$N, n, \tilde{n}$	Number of total/genuine/fake users.
$\mathcal{A}, A, A_i$	Sets of all attributes, target attributes, and the i-th view.
d, m, l, k	Number of total attributes/views/view attributes/target attributes.
D, s_i	The entire domain, number of possible values of i-th attribute.
V_{a_i}, V_A, V_{A_i}	Domain of attribute a_i, value spaces of attribute sets A and A_i.
$n_i, \tilde{n}_i$	Number of genuine and fake users assigned to view A_i.
n^e, n_i^e	Number of estimated genuine users and estimated n_i.
ϵ, p_i, q_i	Privacy budget and FO protocol parameters for view A_i.
$y_i^j, \tilde{y}_i^j$	j-th genuine and crafted reports in view A_i.
$M_A, f_{A_i}^e$	Genuine marginal table and attacker's estimated marginal table.
$\tilde{c}_i, \tilde{c}_i(v)$	Attacker's optimized fake count vector for view A_i and its entry.
$T_A^{\text{tar}}, T_A^{\text{tar}}(v)$	Predefined target marginal table and target frequency for v.
$\tilde{T}_{A_i}, \tilde{T}_{A_i}(v)$	Poisoned initial marginal table for view A_i and its entry for v.
$T_{A_i}^c, T_{A_i}^c(v)$	Poisoned consistent marginal table for view A_i and its entry for v.
$\tilde{T}_A, \tilde{T}_A(v)$	Poisoned final estimated marginal table over A and its entry for v.
$\mathcal{L}_{\text{atk}}, \mathcal{J}_{\text{low}}$	Objective functions of the upper-level and lower-level problems.
$\mathcal{C}_i, \mathcal{C}_i^{\text{GRR}}, \mathcal{C}_i^{\text{OUE}}$	Feasible regions for fake counts in view A_i under GRR/OUE.
$\Delta_A, \Delta_{A_i}, \Pi(\cdot)$	Probability simplices for A and A_i, and Euclidean projection.
P_{ij}, Q_{ij}, H_i, G_i	Binary matrices for marginalizing overlapping views and targets.
$\alpha_i, \beta_{ij}, \gamma_i, \lambda$	Weight parameters in the lower-level optimization problem.
$\mathcal{E}_{\text{loc}}, \mathcal{P}_{\text{CALM}}$	Set of overlapping view pairs and the lower-level problem.
$\xi, \tilde{\mathbf{T}}_t^*, F$	Adjoint variable, lower-level optimal state, and KKT conditions.
$\eta_{\text{in}}, \eta_{\text{out}}$	Learning rates for the lower-level and upper-level optimization.

B. Attack Goal

We consider fine-grained data poisoning attacks [36], [37] against LDP marginal release protocols. The attacker aims to force the final estimated marginal table over the target attribute set A to be as close as possible to a predefined target marginal table. Specifically, let $T_A^{\text{tar}} = (T_A^{\text{tar}}(v))_{v \in V_A}$ be the attacker's predefined target marginal table, and $\tilde{T}_A = (\tilde{T}_A(v))_{v \in V_A}$ be the final estimated marginal table under the attack. The attacker seeks to minimize the attack loss $\mathcal{L}_{\text{atk}}$ as follows:

$$\min \ \mathcal{L}_{\text{atk}} = \left\| \tilde{T}_A - T_A^{\text{tar}} \right\|_2^2 = \sum\nolimits_{v \in V_A} \left(\tilde{T}_A(v) - T_A^{\text{tar}}(v) \right)^2. \tag{4}$$

Here we quantify the attack loss using the squared L_2 distance, which aligns with the Sum of Squared Errors (SSE), a common metric for evaluating the utility of LDP marginal release protocols [26]. Figure 1(b) shows an example of poisoning LDP marginal release. Key notations are summarized in Table I.

IV. Data Poisoning Attack Modeling

A. View Frequencies with Crafted Reports

CALM generates the final estimate via the three phases as introduced in Sec II-C. However, both Phases 2 and 3 are executed on the aggregator's side and the attacker cannot directly manipulate them. The only part that the attacker can directly manipulate is the phase 1, where the attacker can distort the estimated frequencies in the selected views via controlling fake users sending crafted reports. We therefore first model how crafted reports change these initial estimated marginal tables. Specifically, for the i-th view A_i, let $\{y_i^1, \ldots, y_i^{n_i}\}$ be genuine reports and $\{\tilde{y}_i^1, \ldots, \tilde{y}_i^{\tilde{n}_i}\}$ be crafted reports. Let $\tilde{T}_{A_i}$ denote the initial estimated marginal table of view A_i after attack. According to Eq. (1), the estimated frequency of any value combination $v \in V_{A_i}$ in $\tilde{T}_{A_i}$ is calculated as

$$\tilde{T}_{A_i}(v) = \frac{\frac{1}{n_i + \tilde{n}_i}\left(\sum_{j=1}^{n_i} \mathbb{I}_{S(y_i^j)}^{(v)} + \sum_{j=1}^{\tilde{n}_i} \mathbb{I}_{S(\tilde{y}_i^j)}^{(v)} \right) - q_i}{p_i - q_i}, \tag{5}$$

where p_i and q_i are the parameters of the specific FO protocol adopted for view A_i. If it is GRR, then $p_i = \frac{e^\epsilon}{e^\epsilon + |V_{A_i}| - 1}$ and

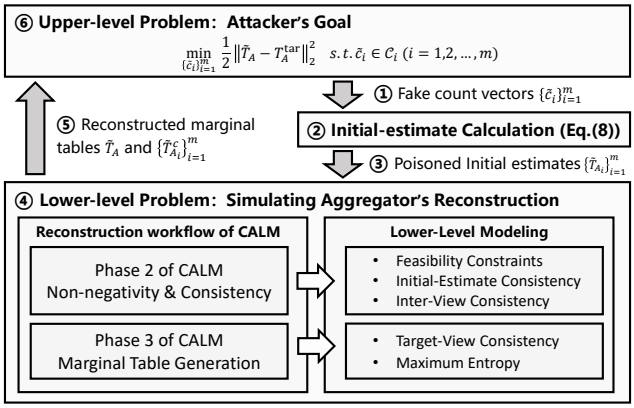


Fig. 2: Overview of the bi-level optimization attack model.

$q_i = \frac{1}{e^\epsilon + |V_{A_i}| - 1}$. If it is OUE, then $p_i = 0.5$ and $q_i = \frac{1}{e^\epsilon + 1}$. Since the genuine reports are locally perturbed, the attacker cannot directly calculate $\sum_{j=1}^{n_i} \mathbb{I}_{S(y_i^j)}^{(v)}$. However, based on the unbiasedness of the FO protocol, the expected value of $\sum_{j=1}^{n_i} \mathbb{I}_{S(y_i^j)}^{(v)}$ can be expressed using the genuine frequency $M_{A_i}(v)$ as $\mathbb{E}[\sum_{j=1}^{n_i} \mathbb{I}_{S(y_i^j)}^{(v)}] = n_i(p_i - q_i)M_{A_i}(v) + n_i q_i$. Estimating the number of genuine users n_i and the genuine frequency $M_{A_i}(v)$ as n_i^e and $f_{A_i}^e(v)$, respectively, we have $\mathbb{E}[\sum_{j=1}^{n_i} \mathbb{I}_{S(y_i^j)}^{(v)}] \approx n_i^e(p_i - q_i)f_{A_i}^e(v) + n_i^e q_i$.

To quantify the impact of the fake users, we define the aggregated fake counts for the value v in view A_i as $\tilde{c}_i(v) = \sum_{j=1}^{\tilde{n}_i} \mathbb{I}_{S(\tilde{y}_i^j)}^{(v)}$. By substituting the expected genuine sum $\mathbb{E}[\sum_{j=1}^{n_i} \mathbb{I}_{S(y_i^j)}^{(v)}]$ and the aggregated fake counts $\tilde{c}_i(v)$ into Eq. (5), we can formulate the frequency after attack as

$$\tilde{T}_{A_i}(v) = \frac{\tilde{c}_i(v) + n_i^e(p_i - q_i)f_{A_i}^e(v) - \tilde{n}_i q_i}{(n_i^e + \tilde{n}_i)(p_i - q_i)}. \tag{6}$$

For simplicity, we express this in a vector format. Let $\tilde{c}_i = (\tilde{c}_i(v))_{v \in V_{A_i}}$ be the fake count vector, and $\tilde{T}_{A_i} = (\tilde{T}_{A_i}(v))_{v \in V_{A_i}}$ be the initial estimated marginal table of view A_i. The vectorization yields

$$\tilde{T}_{A_i} = \frac{\tilde{c}_i + n_i^e(p_i - q_i)f_{A_i}^e - \tilde{n}_i q_i \mathbf{1}_{|V_{A_i}|}}{(n_i^e + \tilde{n}_i)(p_i - q_i)}, \tag{7}$$

where $\mathbf{1}_{|V_{A_i}|}$ is the all-one vector of dimension $|V_{A_i}|$, and $f_{A_i}^e = (f_{A_i}^e(v))_{v \in V_{A_i}}$ is the genuine user's estimated marginal frequency vector for view A_i. As shown above, the initial estimated marginal table for each view are directly parameterized by the fake count vectors $\tilde{c}_i$. Because the final estimate $\tilde{T}_A$ is subsequently reconstructed from initial estimated marginal tables $\tilde{T}_{A_1}, \dots, \tilde{T}_{A_m}$, the attacker can optimize the fake counts to minimize the attack loss $\mathcal{L}_{\text{atk}}$. Formally, the attacker aims to find $\{\tilde{c}_i\}_{i=1}^m$ such that

$$\min_{\{\tilde{c}_i\}_{i=1}^m} \mathcal{L}_{\text{atk}} = \left\|\tilde{T}_A - T_A^{\text{tar}}\right\|_2^2 = \sum_{v \in V_A} \left(\tilde{T}_A(v) - T_A^{\text{tar}}(v)\right)^2. \tag{8}$$

B. Bi-level Optimization Attack Model

Motivation. As discussed in Sec. IV-A, the attacker can craft the fake count vectors $\{\tilde{c}_i\}_{i=1}^m$ to manipulate the initial estimated marginal tables $\{\tilde{T}_{A_i}\}_{i=1}^m$. However, as introduced in Sec. II-C, the aggregator does not output these initial estimates directly. Instead, it processes them through Non-negativity and Consistency (Phase 2) and Marginal Table Generation (Phase 3) to obtain the final estimate $\tilde{T}_A$. Besides, the attacker's goal, i.e., Eq. (4), requires the attacker to minimize the difference between this final estimate $\tilde{T}_A$ and the target marginal table T_A^{tar}. To simultaneously capture the aggregator's reconstruction workflow (i.e., Phases 2 and 3) and achieve the attack goal, we formulate the attack as a bi-level optimization problem, where the lower-level problem simulates the aggregator's reconstruction workflow to derive the final estimate $\tilde{T}_A$, and the upper-level problem uses this $\tilde{T}_A$ to calculate the attack loss $\mathcal{L}_{\text{atk}}$ and optimize the fake counts. Note that unlike standard optimization problems, this problem contains a *nested dependency*. The attack loss cannot be calculated directly from the fake counts $\{\tilde{c}_i\}_{i=1}^m$. Instead, it depends on the final estimate $\tilde{T}_A$, which is the output of the lower-level problem.

Figure 2 illustrates our bi-level optimization framework. Given the fake count vectors $\{\tilde{c}_i\}_{i=1}^m$ (①), the attacker can calculate the initial estimates $\{\tilde{T}_{A_i}\}_{i=1}^m$ via Eq. (7) (②). These estimates then serve as the inputs to the lower-level problem (③). The lower-level problem simulates Phase 2 and Phase 3 of CALM (④) and outputs the reconstructed marginal tables, including the final estimate $\tilde{T}_A$ (⑤). Finally, $\tilde{T}_A$ is passed to the upper-level problem to calculate the attack loss, which in turn guides the optimization of the fake count vectors (⑥). We will formulate the lower-level and upper-level problems in the following, respectively, and provide the complete bi-level optimization problem thereafter.

Lower-level Problem: Aggregator's Reconstruction. In the lower-level problem, the attacker simulate the aggregator's reconstruction workflow. Just as CALM uses the initial estimates $\{T_{A_i}\}_{i=1}^m$ to output the consistent tables $\{T_{A_i}^c\}_{i=1}^m$ and the final estimate T_A, the lower-level problem takes the initial estimates after attack $\{\tilde{T}_{A_i}\}_{i=1}^m$ (which is determined by the fake counts $\{\tilde{c}_i\}_{i=1}^m$) as inputs and aims to optimize the consistent marginal tables after attack $\{\tilde{T}_{A_i}^c\}_{i=1}^m$ and the final estimate after attack $\tilde{T}_A$. We model the aggregator's reconstruction to an optimization problem as follows.

1) Modeling Non-negativity in Phase 2. The aggregator first ensures that no frequency in the marginal tables is smaller than $-\theta_{\text{neg}}$. To model this, we apply feasibility constraints to both $\{\tilde{T}_{A_i}^c\}_{i=1}^m$ and $\tilde{T}_A$, requiring them to be non-negative and to sum to 1. This makes them valid probability distributions and satisfies the lower bound $-\theta_{\text{neg}}$. Specifically, we constrain $\tilde{T}_{A_i}^c$ and $\tilde{T}_A$ to probability simplices $\Delta_{A_i} = \{x \in \mathbb{R}_+^{|V_{A_i}|} \mid \sum_{j=1}^{|V_{A_i}|} x[j] = 1\}$ and $\Delta_A = \{x \in \mathbb{R}_+^{|V_A|} \mid \sum_{j=1}^{|V_A|} x[j] = 1\}$, respectively, where $\mathbb{R}_+$ denotes the set of non-negative real numbers. $x[j]$ denotes the j-th entry of the $|V_{A_i}|$-dimensional (or $|V_A|$-dimensional) vector x.

2) Modeling Consistency in Phase 2. The aggregator also enforces consistency across overlapping views. However, this process involves a topological sort, which is a non-differentiable operation hindering gradient-based optimization. To continuously model this, we introduce the following two consistency, initial-estimate and inter-view consistency.

• ***Initial-Estimate Consistency.*** The consistent marginal tables should retain the information extracted from the initial estimates of users' reports. To formulate this, we introduce a term $\sum_{i=1}^{m} \alpha_i \|\tilde{T}^c_{A_i} - \tilde{T}_{A_i}\|_2^2$ to the lower-level objective function, where α_i is a weight parameter. This ensures that the consistent marginal tables $\{\tilde{T}^c_{A_i}\}_{i=1}^m$ do not deviate excessively from the initial estimates $\{\tilde{T}_{A_i}\}_{i=1}^m$.

• ***Inter-View Consistency.*** The consistent marginal tables of overlapping views must align with each other over their shared attributes. Let $\mathcal{E}_{\text{loc}} = \{(i,j) \mid 1 \le i < j \le m, A_i \cap A_j \neq \emptyset\}$ be the set of overlapping view pairs. For each pair $(i,j) \in \mathcal{E}_{\text{loc}}$, we define two binary marginalization matrices $P_{ij} \in \{0,1\}^{|V_{A_i \cap A_j}| \times |V_{A_i}|}$ and $Q_{ij} \in \{0,1\}^{|V_{A_i \cap A_j}| \times |V_{A_j}|}$, where $V_{A_i \cap A_j}$ is the value space of the intersection of A_i and A_j. Formally, let $w_r \in V_{A_i \cap A_j}$ be the r-th lexicographical value combination in $V_{A_i \cap A_j}$. Let $v_c \in V_{A_i}$ be the c-th lexicographical value combination in V_{A_i}, and $v'_{c'} \in V_{A_j}$ be the c'-th lexicographical value combination in V_{A_j}. The (r,c)-th entry of P_{ij} and the (r,c')-th entry of Q_{ij} are defined as

$$(P_{ij})_{r,c} = \begin{cases} 1, & \text{if } v_c[A_i \cap A_j] = w_r, \\ 0, & \text{otherwise,} \end{cases}$$

$$(Q_{ij})_{r,c'} = \begin{cases} 1, & \text{if } v'_{c'}[A_i \cap A_j] = w_r, \\ 0, & \text{otherwise.} \end{cases}$$

Under this definition, the matrix-vector products $P_{ij}\tilde{T}^c_{A_i}$ and $Q_{ij}\tilde{T}^c_{A_j}$ calculate the marginal distributions of $\tilde{T}^c_{A_i}$ and $\tilde{T}^c_{A_j}$ over their shared attributes $A_i \cap A_j$. Their inconsistency is penalized by $\sum_{(i,j)\in\mathcal{E}_{\text{loc}}} \beta_{ij} \|P_{ij}\tilde{T}^c_{A_i} - Q_{ij}\tilde{T}^c_{A_j}\|_2^2$, where β_{ij} is a weight parameter.

3) Modeling Marginal Table Generation (Phase 3). In phase 3, the aggregator utilizes the Maximum Entropy method with the consistent marginal tables to construct the final estimate. We model it with the following two components:

• ***Target-View Consistency.*** The final marginal table $\tilde{T}_A$ should align with the consistent marginal tables over their shared attributes. To model this, we define marginalization matrices $G_i \in \{0,1\}^{|V_{A\cap A_i}| \times |V_A|}$ and $H_i \in \{0,1\}^{|V_{A\cap A_i}| \times |V_{A_i}|}$, where $V_{A\cap A_i}$ is the value space of the intersection of A and A_i. Let $w_r \in V_{A\cap A_i}$ be the r-th lexicographical value combination in $V_{A\cap A_i}$. Let $v_c \in V_A$ be the c-th lexicographical value combination in V_A, and $v'_{c'} \in V_{A_i}$ be the c'-th lexicographical value combination in V_{A_i}. The (r,c)-th entry of G_i and the (r,c')-th entry of H_i are given by

$$(G_i)_{r,c} = \begin{cases} 1, & \text{if } v_c[A \cap A_i] = w_r, \\ 0, & \text{otherwise,} \end{cases}$$

$$(H_i)_{r,c'} = \begin{cases} 1, & \text{if } v'_{c'}[A \cap A_i] = w_r, \\ 0, & \text{otherwise.} \end{cases}$$

The matrix-vector products $G_i\tilde{T}_A$ and $H_i\tilde{T}^c_{A_i}$ calculate the marginal distributions of the final estimate $\tilde{T}_A$ and the consistent marginal table $\tilde{T}^c_{A_i}$ over their shared attributes $A \cap A_i$. Their inconsistency is penalized by $\sum_{i=1}^{m} \gamma_i \|G_i\tilde{T}_A - H_i\tilde{T}^c_{A_i}\|_2^2$, where γ_i is a weight parameter.

• ***Maximum Entropy.*** Since CALM uses the Maximum Entropy method to generate the final estimate, we add the term $\lambda \sum_{v\in V_A} \tilde{T}_A(v) \log \tilde{T}_A(v)$ to the objective function of lower-level problem, where λ is a weight parameter.

4) The Complete Lower-level Problem. By combining the above components, the aggregator's reconstruction workflow is modeled as the following lower-level optimization problem, denoted as $\mathcal{P}_{\text{CALM}}(\{\tilde{T}_{A_i}\}_{i=1}^m)$:

$$\begin{aligned} \min_{\tilde{T}_A, \{\tilde{T}^c_{A_i}\}_{i=1}^m} \quad & \lambda \sum_{v\in V_A} \tilde{T}_A(v) \log \tilde{T}_A(v) + \sum_{i=1}^{m} \alpha_i \left\| \tilde{T}^c_{A_i} - \tilde{T}_{A_i} \right\|_2^2 \\ + & \sum_{(i,j)\in\mathcal{E}_{\text{loc}}} \beta_{ij} \left\| P_{ij}\tilde{T}^c_{A_i} - Q_{ij}\tilde{T}^c_{A_j} \right\|_2^2 + \sum_{i=1}^{m} \gamma_i \left\| G_i\tilde{T}_A - H_i\tilde{T}^c_{A_i} \right\|_2^2 \\ \text{s.t.} \quad & \tilde{T}_A \in \Delta_A, \quad \tilde{T}^c_{A_i} \in \Delta_{A_i}, \quad i = 1,2,\ldots,m. \end{aligned} \tag{9}$$

In this problem, the initial estimates $\{\tilde{T}_{A_i}\}_{i=1}^m$ (determined by the attacker's fake counts $\{\tilde{c}_i\}_{i=1}^m$) are given, and the attacker aims to find the optimal variables $\tilde{T}_A$ and $\{\tilde{T}^c_{A_i}\}_{i=1}^m$.

Upper-level Problem: Attacker's Goal. In the upper-level problem, the attacker optimizes the fake count vectors $\{\tilde{c}_i\}_{i=1}^m$ to minimize the attack loss. To facilitate gradient computation during optimization, we scale the squared L_2 distance by a constant factor of $1/2$. This convention simplifies the subsequent derivative expressions without altering the optimal solution. Thus, the upper-level objective is to minimize $\frac{1}{2}\|\tilde{T}_A - T_A^{\text{tar}}\|_2^2$, where $\tilde{T}_A$ is the final marginal table and T_A^{tar} is the target marginal table. However, the fake count vectors cannot be arbitrary. They must comply with the reporting format of the FO protocol assigned to each view. If view A_i uses GRR, each user submits a categorical item. Thus, the sum of the fake counts across all possible values in V_{A_i} must equal the total number of fake users $\tilde{n}_i$ assigned to this view. The feasible region is $\mathcal{C}_i^{\text{GRR}} = \{x \in \mathbb{R}_+^{|V_{A_i}|} \mid \sum_{j=1}^{|V_{A_i}|} x[j] = \tilde{n}_i\}$, where $x[j]$ represents the j-th entry of the vector x. If view A_i adopts OUE, users submit binary strings where each bit is perturbed independently. Consequently, the number of "1"s reported for any specific value must be bounded by the number of fake users $\tilde{n}_i$ and must be non-negative. Thus, the feasible region is $\mathcal{C}_i^{\text{OUE}} = \{x \in \mathbb{R}_+^{|V_{A_i}|} \mid 0 \le x[j] \le \tilde{n}_i,\ \forall j \in \{1,\ldots,|V_{A_i}|\}\}$. We denote the feasible region for the i-th view as $\mathcal{C}_i$, which is either $\mathcal{C}_i^{\text{GRR}}$ or $\mathcal{C}_i^{\text{OUE}}$ depending on the mechanism chosen by the Ada protocol.

The Complete Bi-level Optimization Problem. By consolidating the upper-level and lower-level problems, we formulate the attack as the following bi-level optimization problem:

$$\begin{aligned} \min_{\{\tilde{c}_i\}_{i=1}^m} \quad & \frac{1}{2} \left\| \tilde{T}_A - T_A^{\text{tar}} \right\|_2^2 \\ \text{s.t.} \quad & \tilde{c}_i \in \mathcal{C}_i, \quad i = 1,2,\ldots,m, \\ & \left(\tilde{T}_A, \{\tilde{T}^c_{A_i}\}_{i=1}^m\right) \in \mathcal{P}_{\text{CALM}}\left(\{\tilde{T}_{A_i}\}_{i=1}^m\right). \end{aligned} \tag{10}$$

Eq. (10) shows the nested dependency of the attack. The attacker controls the variables $\tilde{c}_i$, which are bounded by the protocol's feasibility constraints $\mathcal{C}_i$. These fake counts modify the initial estimated marginal tables $\{\tilde{T}_{A_i}\}_{i=1}^m$, which serve as the input to the aggregator's reconstruction workflow modeled by the lower-level problem $\mathcal{P}_{\text{CALM}}$. The solution to the lower-level problem, i.e., the estimated marginal table $\tilde{T}_A$, directly dictates the upper-level attack loss.

V. Solving the Bi-level Optimization Problem

The upper-level objective in our attack model depends on the optimal solution of the lower-level aggregator's reconstruction problem. Because of this nested dependency, the gradients of the attack loss with respect to the fake count vectors $\{\tilde{c}_i\}_{i=1}^m$ cannot be directly calculated, which makes standard optimization methods like gradient descent inapplicable. To tackle this, in this section, we propose an iterative bi-level optimization algorithm. Specifically, in the t-th iteration of the bi-level optimization, the attacker first solves the lower-level problem to obtain the final estimated marginal table and consistent marginal tables corresponding to the current fake counts. Subsequently, the attacker utilizes implicit gradients to update the upper-level fake count vectors. In the following sections, we detail this bi-level optimization algorithm.

A. Solving the Lower-level Problem

We first solve the lower-level problem at the t-th iteration of the bi-level optimization. Given the fake count vectors $\{\tilde{c}_i^{(t)}\}_{i=1}^m$ at the t-th iteration, the attacker can calculate the initial estimated marginal tables for each view, denoted as $\tilde{T}_{A_i}^{(t)}$, according to Eq. (7). These tables serve as the inputs of the lower-level problem and are fixed during the lower-level optimization. Let $\mathcal{J}_{\text{low}}(\tilde{T}_A, \{\tilde{T}_{A_i}^c\}_{i=1}^m)$ denote the objective function of the lower-level problem in Eq. (9). Since the lower-level objective $\mathcal{J}_{\text{low}}$ is differentiable with respect to the final estimated marginal table $\tilde{T}_A$ and the consistent marginal tables $\{\tilde{T}_{A_i}^c\}_{i=1}^m$, we can calculate their exact gradients. Specifically, the gradient of $\mathcal{J}_{\text{low}}$ with respect to $\tilde{T}_A$ is derived as

$$\nabla_{\tilde{T}_A}\mathcal{J}_{\text{low}} = 2\sum_{i=1}^{m}\gamma_i G_i^\top\left(G_i\tilde{T}_A - H_i\tilde{T}_{A_i}^c\right) + \lambda\left(\log\tilde{T}_A + \mathbf{1}_{|V_A|}\right), \tag{11}$$

where $\log\tilde{T}_A$ denotes the element-wise natural logarithm, and $\mathbf{1}_{|V_A|}$ is an all-one vector of dimension $|V_A|$. Similarly, for any view index $i \in \{1,\dots,m\}$, the gradient of $\mathcal{J}_{\text{low}}$ with respect to the consistent marginal table $\tilde{T}_{A_i}^c$ can be calculated as

$$\begin{aligned}\nabla_{\tilde{T}_{A_i}^c}\mathcal{J}_{\text{low}} &= 2\alpha_i\left(\tilde{T}_{A_i}^c - \tilde{T}_{A_i}^{(t)}\right) + 2\sum_{(i,j)\in\mathcal{E}_{\text{loc}}}\beta_{ij}P_{ij}^\top\left(P_{ij}\tilde{T}_{A_i}^c - Q_{ij}\tilde{T}_{A_j}^c\right)\\ &+2\sum_{(j,i)\in\mathcal{E}_{\text{loc}}}\beta_{ji}Q_{ji}^\top\left(Q_{ji}\tilde{T}_{A_i}^c - P_{ji}\tilde{T}_{A_j}^c\right) + 2\gamma_i H_i^\top\left(H_i\tilde{T}_{A_i}^c - G_i\tilde{T}_A\right).\end{aligned} \tag{12}$$

Because both $\tilde{T}_A$ and $\{\tilde{T}_{A_i}^c\}_{i=1}^m$ represent valid probability distributions, they must strictly reside within their respective probability simplices Δ_A and Δ_{A_i}. Thus, we employ a joint Projected Gradient Descent (PGD) method to solve for $\tilde{T}_A$ and $\{\tilde{T}_{A_i}^c\}_{i=1}^m$. Unlike standard gradient descent, in each iteration t_p of the PGD method, $\tilde{T}_A$ and $\{\tilde{T}_{A_i}^c\}_{i=1}^m$ are updated as

$$\tilde{T}_A^{(t_p+1)} = \Pi_{\Delta_A}\left(\tilde{T}_A^{(t_p)} - \eta_{\text{in}}\nabla_{\tilde{T}_A}\mathcal{J}_{\text{low}}^{(t_p)}\right), \tag{13}$$

$$\tilde{T}_{A_i}^{c,(t_p+1)} = \Pi_{\Delta_{A_i}}\left(\tilde{T}_{A_i}^{c,(t_p)} - \eta_{\text{in}}\nabla_{\tilde{T}_{A_i}^c}\mathcal{J}_{\text{low}}^{(t_p)}\right). \tag{14}$$

where $\eta_{\text{in}} > 0$ is the learning rate and $\mathcal{J}_{\text{low}}^{(t_p)}$ is the lower-level objective function evaluated at the t_p-th iteration. $\Pi_{\Delta_A}(x)$ denotes the Euclidean projection onto the probability simplex Δ_A, which maps any arbitrary vector x to the closest valid probability distribution (i.e., finding a vector in Δ_A that minimizes the Euclidean distance to x and ensures all elements are non-negative and sum to one).

Algorithm 1: Solving the Lower-level Problem

Input: Current fake count vectors $\{\tilde{c}_i^{(t)}\}_{i=1}^m$; Number of fake users $\{\tilde{n}_i\}_{i=1}^m$; Attacker's estimates $\{n_i^e, f_{A_i}^e\}_{i=1}^m$; FO parameters $\{p_i, q_i\}_{i=1}^m$; weight parameters $\alpha_i, \gamma_i, \beta_{ij}, \lambda$; Learning rate η_{in}; Maximum iterations S_{in}; Convergence threshold τ_{in}.

Output: Optimized marginal tables $\tilde{T}_A^*$ and $\{\tilde{T}_{A_i}^{c,*}\}_{i=1}^m$.

1 Calculate the initial marginal tables $\{\tilde{T}_{A_i}^{(t)}\}_{i=1}^m$ with Eq. (7);
2 Initialize $\tilde{T}_A^{(0)} \in \Delta_A$ and $\tilde{T}_{A_i}^{c,(0)} \in \Delta_{A_i}$ for $i = 1, 2, \dots, m$;
3 **for** $t_p = 0, 1, \dots, S_{\text{in}} - 1$ **do**
4 Calculate the gradients of $\mathcal{J}_{\text{low}}$ with respect to $\tilde{T}_A$ and $\{\tilde{T}_{A_i}^c\}_{i=1}^m$ via Eq. (11) and Eq. (12);
5 Update the final estimated marginal table via the PGD method: $\tilde{T}_A^{(t_p+1)} = \Pi_{\Delta_A}\left(\tilde{T}_A^{(t_p)} - \eta_{\text{in}}\nabla_{\tilde{T}_A}\mathcal{J}_{\text{low}}^{(t_p)}\right)$;
6 **for** $i = 1, 2, \dots, m$ **do**
7 Update the consistent marginal table for the i-th view via the PGD method: $\tilde{T}_{A_i}^{c,(t_p+1)} = \Pi_{\Delta_{A_i}}\left(\tilde{T}_{A_i}^{c,(t_p)} - \eta_{\text{in}}\nabla_{\tilde{T}_{A_i}^c}\mathcal{J}_{\text{low}}^{(t_p)}\right)$;
8 **end**
9 **if** $\|\tilde{T}_A^{(t_p+1)} - \tilde{T}_A^{(t_p)}\|_2^2 + \sum_{i=1}^m \|\tilde{T}_{A_i}^{c,(t_p+1)} - \tilde{T}_{A_i}^{c,(t_p)}\|_2^2 \le \tau_{\text{in}}$ **then**
10 Break;
11 **end**
12 **end**
13 Set $\tilde{T}_A^* \leftarrow \tilde{T}_A^{(t_p+1)}$ and $\tilde{T}_{A_i}^{c,*} \leftarrow \tilde{T}_{A_i}^{c,(t_p+1)}$ for $i = 1, 2, \dots, m$;
14 **return** $\tilde{T}_A^*$ and $\{\tilde{T}_{A_i}^{c,*}\}_{i=1}^m$;

Algorithm 1 summarizes the process of solving the lower-level problem. The algorithm first calculates the initial estimated marginal tables $\{\tilde{T}_{A_i}^{(t)}\}_{i=1}^m$ using Eq. (7) (Line 1). It then initializes the final estimated marginal table $\tilde{T}_A^{(0)}$ and all consistent marginal tables $\{\tilde{T}_{A_i}^{c,(0)}\}_{i=1}^m$ as valid distributions within their respective probability simplices (Line 2). Next, the algorithm enters an iterative loop to perform the joint Projected Gradient Descent (Lines 3–12). In each iteration t_p, the algorithm evaluates the exact gradients of the objective function $\mathcal{J}_{\text{low}}$ using Eq. (11) and Eq. (12) based on the initial estimated marginal tables (Line 4). Using these calculated gradients, it applies gradient descent steps followed by Euclidean projections Π_{Δ_A} and $\Pi_{\Delta_{A_i}}$ to update the final estimated marginal table and the consistent marginal tables for all m views (Lines 5–8). After the updates, the algorithm calculates the total squared L_2 norm difference of the variables between the current iteration t_p+1 and the previous iteration t_p. If this difference falls below a predefined tolerance threshold τ_{in}, the loop will be stopped and the algorithm will return the latest optimized variables as the final marginal table $\tilde{T}_A^*$ and the consistent marginal tables $\{\tilde{T}_{A_i}^{c,*}\}_{i=1}^m$ (Lines 9–14).

B. Implicit Gradient Calculation

After solving the lower-level problem, the attacker obtains the optimal reconstructed variables. We denote these variables at the t-th iteration of the bi-level optimization as $\tilde{\mathbf{T}}_t^* = (\tilde{T}_{A,t}^*, \{\tilde{T}_{A_i,t}^{c,*}\}_{i=1}^m)$. Following Eq. (10), the upper-level attack loss is evaluated as $\mathcal{L}_{\text{atk}}^{(t)} = \frac{1}{2}\|\tilde{T}_{A,t}^* - T_A^{\text{tar}}\|_2^2$. To minimize this loss, the attacker must calculate the gradients of $\mathcal{L}_{\text{atk}}^{(t)}$

with respect to the fake count vectors $\tilde{c}_i^{(t)}$, i.e., $\nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)}$.

However, since $\tilde{\mathbf{T}}_t^*$ is implicitly derived from the iterative lower-level optimization, it lacks an explicit analytical form. We address this by employing implicit differentiation. Denote the collection of fake count vectors as $\tilde{\mathbf{c}}^{(t)} = \{\tilde{c}_i^{(t)}\}_{i=1}^m$. Assuming the optimal solution $\tilde{\mathbf{T}}_t^*$ lies in the relative interior of the probability simplices Δ_A and Δ_{A_i}, the non-negativity constraints are strictly satisfied. Thus, the Lagrangian of the lower-level problem, denoted by $\mathcal{J}(\tilde{\mathbf{T}}_t^*, \kappa_t; \tilde{\mathbf{c}}^{(t)})$, only needs to incorporate the sum-to-one equality constraints:

$$\mathcal{J}\left(\tilde{\mathbf{T}}_t^*, \kappa_t; \tilde{\mathbf{c}}^{(t)}\right) = \mathcal{J}_{\text{low}} + \kappa_{A,t}\left(\mathbf{1}_{|V_A|}^\top \tilde{T}_{A,t}^* - 1\right) + \sum_{i=1}^m \kappa_{i,t}\left(\mathbf{1}_{|V_{A_i}|}^\top \tilde{T}_{A_i,t}^{c,*} - 1\right), \quad (15)$$

where $\kappa_t = (\kappa_{A,t}, \kappa_{1,t}, \ldots, \kappa_{m,t})$ are the Lagrangian multipliers, and $\mathbf{1}_D$ denotes an all-one vector of dimension D. Because $\tilde{\mathbf{T}}_t^*$ is optimal, it must satisfy the Karush–Kuhn–Tucker (KKT) conditions. These conditions define a system of equations $F(\tilde{\mathbf{T}}_t^*, \kappa_t; \tilde{\mathbf{c}}^{(t)}) = 0$, given by

$$F\left(\tilde{\mathbf{T}}_t^*, \kappa_t; \tilde{\mathbf{c}}^{(t)}\right) = \begin{bmatrix} \nabla_{\tilde{\mathbf{T}}} \mathcal{J}\left(\tilde{\mathbf{T}}_t^*, \kappa_t; \tilde{\mathbf{c}}^{(t)}\right)^\top \\ \mathbf{1}_{|V_A|}^\top \tilde{T}_{A,t}^* - 1 \\ \vdots \\ \mathbf{1}_{|V_{A_m}|}^\top \tilde{T}_{A_m,t}^{c,*} - 1 \end{bmatrix}. \quad (16)$$

Based on this KKT system, we can differentiate F with respect to the attacker's fake count variables to obtain the gradients $\nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)}$. It is summarized in the following theorem.

Theorem 1 (Implicit Gradient)**.** *Given the optimal lower-level KKT system $F(\tilde{\mathbf{T}}_t^*, \kappa_t; \tilde{\mathbf{c}}^{(t)}) = 0$, the gradient of the attack loss $\mathcal{L}_{\text{atk}}^{(t)}$ with respect to the fake count vector $\tilde{c}_i^{(t)}$ is*

$$\nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)} = -\frac{1}{(n_i^e + \tilde{n}_i)(p_i - q_i)} \left(\frac{\partial F}{\partial \tilde{T}_{A_i}^{(t)}}\right)^\top \xi, \quad (17)$$

where the adjoint variable ξ is the solution to the linear system $\left(\frac{\partial F}{\partial (\tilde{\mathbf{T}}_t^*, \kappa_t)}\right)^\top \xi = \nabla_{(\tilde{\mathbf{T}}_t^*, \kappa_t)} \mathcal{L}_{\text{atk}}^{(t)}$. $\partial F / \partial \tilde{T}_{A_i}^{(t)}$ is calculated as

$$\frac{\partial F}{\partial \tilde{T}_{A_i}^{(t)}} = \begin{bmatrix} \mathbf{0}_{\left(|V_A| + \sum_{j=1}^{i-1} |V_{A_j}|\right) \times |V_{A_i}|}^\top \\ \left(-2\alpha_i I_{|V_{A_i}|}\right)^\top \\ \mathbf{0}_{\left(\sum_{j=i+1}^m |V_{A_j}| + m + 1\right) \times |V_{A_i}|}^\top \end{bmatrix}.$$

where $\mathbf{0}_{D_1 \times D_2}$ is an all-zero matrix of dimension $D_1 \times D_2$, and $I_{|V_{A_i}|}$ is a $|V_{A_i}| \times |V_{A_i}|$ identity matrix.

Proof. See Appendix A1. □

After obtaining the gradients $\nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)}$, the attacker can update the fake count vectors by projected gradient descent:

$$\tilde{c}_i^{(t+1)} = \Pi_{\mathcal{C}_i}\left(\tilde{c}_i^{(t)} - \eta_{\text{out}} \nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)}\right), \quad i = 1, 2, \ldots, m, \quad (18)$$

where η_{out} is the learning rate of the upper-level optimization, and $\Pi_{\mathcal{C}_i}(\cdot)$ denotes the Euclidean projection onto $\mathcal{C}_i$. If GRR is used, $\mathcal{C}_i = \mathcal{C}_i^{\text{GRR}}$. If OUE is used, $\mathcal{C}_i = \mathcal{C}_i^{\text{OUE}}$.

C. Overall Bi-level Optimization Algorithm

Algorithm 2 summarizes the process to solve the bi-level problem. The algorithm begins by randomly initializing the fake count vectors $\tilde{c}_i^{(0)}$ within the feasible regions $\mathcal{C}_i$ (Line 1).

Algorithm 2: Solving the Bi-level Problem

Input: Target marginal table T_A^{tar}; Number of fake users $\{\tilde{n}_i\}_{i=1}^m$; Attacker's estimates $\{n_i^e, f_{A_i}^e\}_{i=1}^m$; FO parameters $\{p_i, q_i\}_{i=1}^m$; Learning rate η_{out}; Maximum iterations S_{out}; Convergence threshold τ_{out}; Lower-level parameters $\alpha_i, \gamma_i, \beta_{ij}, \lambda, \eta_{\text{in}}, S_{\text{in}}, \tau_{\text{in}}$.
Output: Optimized fake count vectors $\{\tilde{c}_i^*\}_{i=1}^m$.

1 Initialize fake count vectors $\tilde{c}_i^{(0)} \in \mathcal{C}_i$ for $i = 1, 2, \ldots, m$;
2 **for** $t = 0, 1, \ldots, S_{\text{out}} - 1$ **do**
3 Call Algorithm 1 with $\{\tilde{c}_i^{(t)}\}_{i=1}^m$ to obtain $\tilde{T}_{A,t}^*$ and $\{\tilde{T}_{A_i,t}^{c,*}\}_{i=1}^m$;
4 Calculate the attack loss $\mathcal{L}_{\text{atk}}^{(t)} = \frac{1}{2}\left\|\tilde{T}_{A,t}^* - T_A^{\text{tar}}\right\|_2^2$;
5 **for** $i = 1, 2, \ldots, m$ **do**
6 Calculate the implicit gradient $\nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)}$ via Eq. (17);
7 Update the fake count vector via the PGD method: $\tilde{c}_i^{(t+1)} = \Pi_{\mathcal{C}_i}\left(\tilde{c}_i^{(t)} - \eta_{\text{out}} \nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)}\right)$;
8 **end**
9 **if** $\sum_{i=1}^m \left\|\tilde{c}_i^{(t+1)} - \tilde{c}_i^{(t)}\right\|_2^2 \le \tau_{\text{out}}$ **then**
10 Break;
11 **end**
12 **end**
13 Set $\tilde{c}_i^* \leftarrow \tilde{c}_i^{(t+1)}$ for $i = 1, 2, \ldots, m$;
14 **return** $\{\tilde{c}_i^*\}_{i=1}^m$;

It then enters the main iterative optimization loop (Lines 2–14). In each iteration t, the algorithm first invokes Algorithm 1 by providing the current fake counts $\{\tilde{c}_i^{(t)}\}_{i=1}^m$ as input (Line 3). Algorithm 1 calculates the initial estimated tables and returns the lower-level variables $\tilde{T}_{A,t}^*$ and $\{\tilde{T}_{A_i,t}^{c,*}\}_{i=1}^m$. Leveraging these reconstructed variables, the attacker evaluates the current upper-level attack loss (Line 4). Then, for each fake count vector $\tilde{c}_i^{(t)}$ ($i \in \{1, \ldots, m\}$), the algorithm calculates the gradients $\nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)}$ according to Theorem 1 (Line 6) and applies the projected gradient descent step via Eq. (18) to yield the updated fake counts $\tilde{c}_i^{(t+1)}$ (Line 7).After that, the algorithm evaluates the difference of the fake counts between consecutive iterations (Line 9). If the difference is smaller than the predefined threshold τ_{out}, the iterative loop will be terminated. Finally, the algorithm returns the optimized fake count vectors $\{\tilde{c}_i^*\}_{i=1}^m$ (Line 14).

D. Crafting Malicious Reports for Fake Users

To finalize the attack, the optimized continuous vectors $\{\tilde{c}_i^*\}_{i=1}^m$ must be converted into discrete reports $\{\tilde{y}_i^1, \ldots, \tilde{y}_i^{\tilde{n}_i}\}$ for each view A_i $\forall i \in \{1, \ldots, m\}$. To achieve this, for each view A_i, the attacker first rounds the optimized fake count vector $\tilde{c}_i^* = (\tilde{c}_i^*(v))_{v \in V_{A_i}}$ to an integer vector $\hat{c}_i^* = (\hat{c}_i^*(v))_{v \in V_{A_i}}$ while strictly maintaining the feasibility constraints $\mathcal{C}_i$.If view A_i adopts GRR, each report $\tilde{y}_i^j$ is a single value v from V_{A_i}. The attacker assigns these reports such that the number of fake users reporting v is equal to $\hat{c}_i^*(v)$ for $\forall v \in V_{A_i}$.If OUE is adopted, each report $\tilde{y}_i^j$ is a binary string of length $|V_{A_i}|$. For the r-th lexicographical value combination $w_r \in V_{A_i}$, the attacker constructs these strings bit by bit such that the sum of the r-th bits across fake users in view A_i, i.e., $\sum_{j=1}^{\tilde{n}_i} \tilde{y}_i^j[r]$, matches the integer count $\hat{c}_i^*(w_r)$. These crafted reports $\{\tilde{y}_i^j\}_{j=1}^{\tilde{n}_i}$ are then directly sent to the aggregator, steering the final estimate toward the target T_A^{tar}.

E. Theoretical Analysis

Lower Bound on Attack Loss. We characterize the lower bound on the attack loss induced by CALM's reconstruction.

Theorem 2. *Let* $\mathcal{N} = \{u \in \mathbb{R}^{|V_A|} \mid \mathbf{1}^\top u = 0,\ G_i u = 0,\ \forall i\}$ *and* $r = \max_{u \in \mathcal{N}, \|u\|_2 \le 1} |u^\top \log T_A^{\text{tar}}|$. *Suppose* $\lambda > 0$ *and* $T_A^{\text{tar}}(v) > 0$ *for all* $v \in V_A$. *For any feasible fake count vectors, the attack loss satisfies* $\mathcal{L}_{\text{atk}} \ge \frac{1}{2}\left(\min_{v \in V_A} T_A^{\text{tar}}(v)\right)^2 \left(\frac{r}{1+r}\right)^2$.

Proof. See Appendix A2. □

Computational Cost. Calculating the lower-level gradients once requires $O(m|V_A| + \sum_{i=1}^{m} |V_{A_i}| + \sum_{(i,j) \in \mathcal{E}_{\text{loc}}} (|V_{A_i}| + |V_{A_j}|))$ time. Let S_{in} be the number of lower-level iterations, S_{adj} be the number of iterations for solving the adjoint variable ξ in implicit differentiation, and S_{out} be the number of iterations for solving the overall bi-level problem. Then the total computational cost is $O(S_{\text{out}}(S_{\text{in}} + S_{\text{adj}})(m|V_A| + \sum_{i=1}^{m} |V_{A_i}| + \sum_{(i,j) \in \mathcal{E}_{\text{loc}}} (|V_{A_i}| + |V_{A_j}|)))$.

VI. Attacking Other LDP Protocols

Our proposed attack can be generalized to other mainstream LDP marginal release protocols, including FC [10], AM [22], [23], FT [24], and EM [25] methods. Specifically, we formulate single-level optimization problems to attack FC, AM, and FT and a bi-level optimization problem to attack EM. Due to page limits, we provide the details in Appendix B.

VII. Experimental Evaluation

A. Experimental Setup

Datasets. We use the following four datasets as existing studies on LDP marginal release [26], [24]. BMS-POS [42] is a dataset containing merchant transactions of half a million users. Kosarak [43] is a dataset of click streams on Hungarian website that contains about one million users. Adult [44] is from UCI ML repository, which contains around 50 thousands records. BFive [45] contains around 1 million records of personality survey data for Big Five personality traits.

Target Marginal Tables. We set the target marginal table T_A^{tar} to follow four typical distributions. Specifically, we sort all value combinations $v \in V_A$ lexicographically and assign the target frequencies $T_A^{\text{tar}}(v)$ based on discretized probability density functions of the following distributions:

- Uniform: T_A^{tar} follows a uniform distribution, with all target frequencies $T_A^{\text{tar}}(v)$ equal to $1/|V_A|$.
- Gaussian: T_A^{tar} follows a discretized Gaussian distribution, peaking at the median lexicographical combination and symmetrically decreasing toward both ends.
- Exponential: T_A^{tar} follows a discretized exponential distribution. It peaks at the first lexicographical combination and monotonically decays as the lexicographical index increases.
- Bimodal: T_A^{tar} is generated by a mixture of two Gaussian distributions, forming peaks in the mid-left and mid-right regions of the sorted lexicographical domain.

Metric. Consistent with our attack goal in Eq. (4) and existing studies on LDP marginal release [26], we assess the performance of attacks with the Sum of Squared Errors

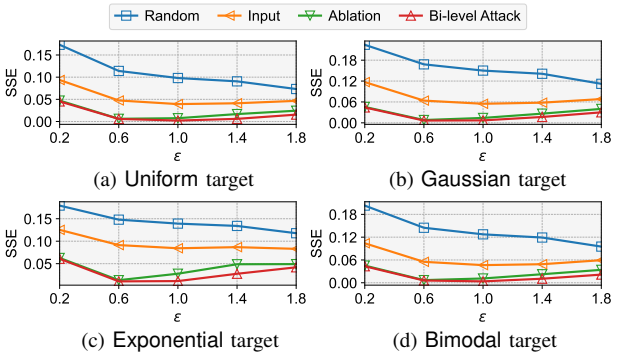


Fig. 3: Attack performance on different target marginal tables, varying ϵ (BMS-POS dataset).

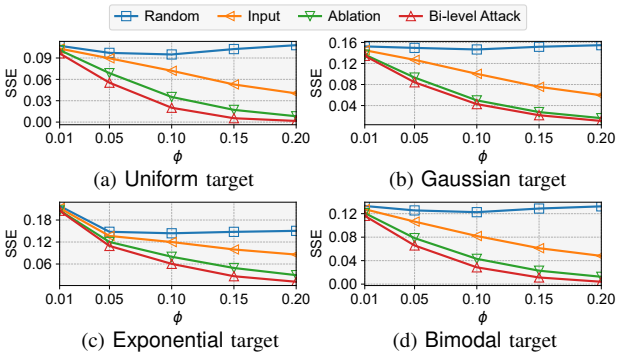


Fig. 4: Attack performance on different target marginal tables, varying ϕ (Adult dataset).

$SSE = \sum_{v \in V_A} (\tilde{T}_A(v) - T_A^{\text{tar}}(v))^2$, where $\tilde{T}_A$ is the final estimate after attack and T_A^{tar} is the target marginal table.

Parameter Settings. By default, we set the privacy budget to $\epsilon = 1$ and the fake user ratio to $\phi = \tilde{n}/N = 0.1$. The attacker estimates the genuine data distribution f^e by snooping 5% of the genuine users' private data, and approximates the genuine user count n^e using round numbers of the exact count n [36]. Besides, the weight parameters in the lower-level problem ($\alpha_i, \beta_{ij}, \gamma_i$, and λ) are set to 1. We set the total number of users to $N = 2^{16}$, the total number of attributes to $d = 8$, and the target attribute set size to $k = |A| = 4$. Following [26], we set the domain size of each attribute to $s_i = 2\ \forall i \in \{1, ..., d\}$ for BMS-POS and Kosarak datasets, and $s_i = 3\ \forall i \in \{1, ..., d\}$ for Adult and BFive datasets.

Compared Algorithms. Due to no prior attacks, we evaluate our attack (denoted as Bi-level Attack) with three baselines:

- **Random:** Fake users submit completely random reports. It is a naive baseline to show the impact of data injection.
- **Input:** Following [36], [37], this baseline optimizes the fake users' input data instead of the perturbed data. These crafted inputs must undergo LDP perturbations before submission.
- **Ablation:** To verify the necessity of bi-level optimization, this baseline manipulates the fake counts to force the initial estimates to directly match corresponding target frequencies, completely ignoring the aggregator's reconstruction workflow.

All experiments are conducted on a computer equipped with an Intel Core i7-12700H CPU and 16 GB of RAM.

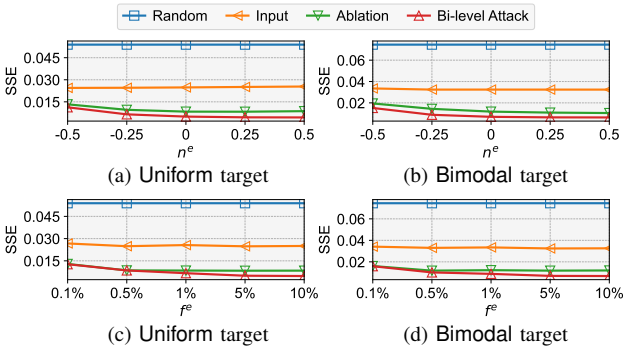
B. Experimental Results

Main Results. Figures 3 and 4 illustrate the attack performance with different ϵ and ϕ, respectively. Overall, our proposed attack (i.e., Bi-level Attack) achieves the lowest SSE and outperforms all baselines. The Random baseline performs the worst, as the fake users submit random reports to the aggregator without any optimization. The Input baseline achieves a lower SSE than Random, indicating its capability to manipulate the final estimate. However, it is still inferior to Ablation and Bi-level Attack because it requires the crafted data to undergo LDP perturbation before submission, which injects massive noise. Although the Ablation baseline bypasses the LDP perturbation and directly sends crafted data to the aggregator, it completely ignores the aggregator's reconstruction workflow. Thus, even if Ablation successfully forces the initial estimates to match the target marginal table, its attack performance is suppressed during CALM's subsequent reconstruction, making it inferior to Bi-level Attack.

Impact of ϵ. Figure 3 illustrates the impact of the privacy budget ϵ on the attack performance. Unsurprisingly, Bi-level Attack consistently achieves the best attack performance. Overall, as ϵ increases, the noise injected by the LDP protocol decreases, which generally leads to a reduction in the SSE. However, when ϵ becomes relatively large, we observe a slight increase in the SSE. This is because, as ϵ increases, the term $p_i - q_i$ in the denominator of Eq. (5) gradually approaches 1 from a value less than 1. This diminishes the impact of the fake users' crafted data on the initial estimates, thereby weakening the attacker's capability to manipulate the final estimate $\tilde{T}_A$. Despite this, the Bi-level Attack still maintains the best attack performance among all compared methods.

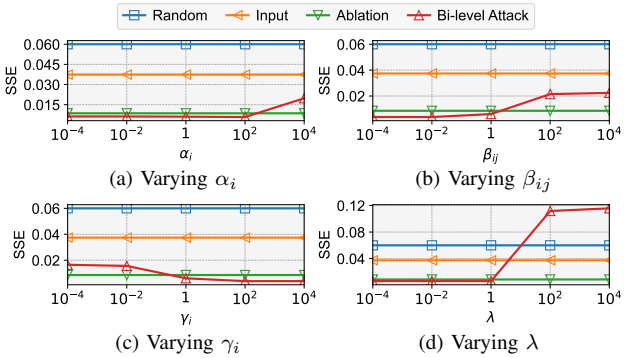
Impact of ϕ. Figure 4 presents the impact of the fake user ratio ϕ on the attack performance. In general, a larger ϕ means that more fake users participate in the poisoning attack, leading to a smaller SSE. However, the SSE of the Random baseline remains basically unchanged or even slightly increases as ϕ increases. This is because the fake users in the Random baseline merely send random data to the aggregator. Injecting more fake users simply adds more random noise, which does not steer the final estimate $\tilde{T}_A$ closer to the target marginal table. In contrast, Bi-level Attack successfully leverages the additional fake users to further minimize the SSE. Even with only 1% fake users, it still achieves lower SSE than all baselines across the four different targets.

Impact of Attacker's Estimations. Figure 5 shows the impact of the attacker's estimations on the attack performance. Specifically, the estimated number of genuine users n^e deviates from the exact count n by $\pm 25\%$ and $\pm 50\%$, while the estimated genuine distribution f^e is estimated by snooping 0.1% to 10% of genuine users. Generally, more accurate estimations of n^e and f^e yield a smaller SSE, indicating better attack performance. We also observe that Bi-level Attack is more sensitive to these estimations than the baselines. This is because Bi-level Attack relies on the lower-level problem to simulate the aggregator's reconstruction workflow. And the inputs of this simulation, i.e., the initial estimated tables, are directly parameterized by n^e and f^e. If these estimations are inaccurate, the simulated final estimate will fail to align with the actual one, thereby reducing the performance of the attack.



(a) Uniform target (b) Bimodal target (c) Uniform target (d) Bimodal target

Fig. 5: Attack performance with different n^e (the first row) and f^e (the second row) on Adult dataset.



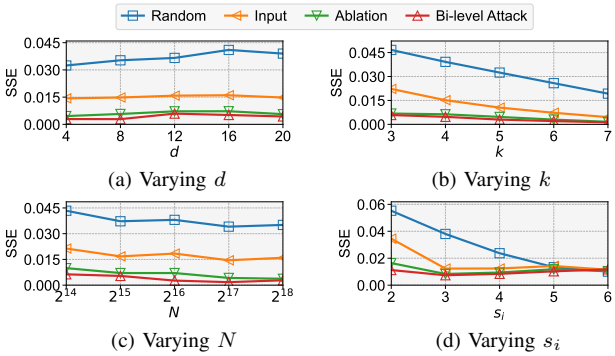
(a) Varying α_i (b) Varying β_{ij} (c) Varying γ_i (d) Varying λ

Fig. 6: Attack performance with different weight parameters (BFive dataset with Exponential target).

Impact of Weight Parameters. Figure 6 shows the impact of the weight parameters ($\alpha_i, \beta_{ij}, \gamma_i$ and λ) in the lower-level problem. Note that the SSEs of the baselines remain constant, as they do not model the lower-level reconstruction workflow. For Bi-level Attack, the SSE increases rapidly with large α_i, β_{ij}, or λ, or with a small γ_i. This is because γ_i controls the Target-View Consistency, linking the final estimate $\tilde{T}_A$ to the consistent marginal tables $\{\tilde{T}^c_{A_i}\}_{i=1}^m$. When γ_i is too small, or relatively diminished by large α_i, β_{ij}, and λ, the final estimate $\tilde{T}_A$ becomes decoupled from the initial estimates. Thus, the implicit gradients cannot effectively guide the optimization of fake counts, which degrades the attack performance.

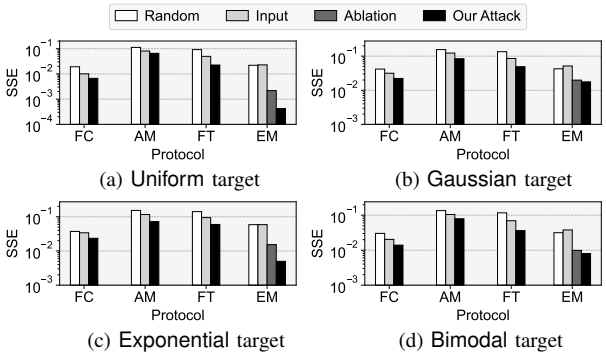
Impact of d. Figure 7(a) illustrates the impact of the total number of attributes d. Overall, as d increases, the SSE of all methods exhibits a slight increase. This is because a larger dimensionality d requires the aggregator to construct a larger number of views m to adequately cover the entire attribute space. Since CALM uniformly partitions the total users into m disjoint groups, an increased m results in fewer users (both genuine and fake) allocated to each view, amplifying the estimation variance in the initial estimated marginal tables and introducing higher noise into the final estimate $\tilde{T}_A$.

Impact of k. Figure 7(b) presents the impact of the target attribute set size k. As k increases, the SSE of all attacks decreases. While high-dimensional marginal release typically



(a) Varying d (b) Varying k (c) Varying N (d) Varying s_i

Fig. 7: Attack performance with different d, k, N and s_i (BFive dataset with Exponential target).



(a) Uniform target (b) Gaussian target (c) Exponential target (d) Bimodal target

Fig. 8: Attack performance on other LDP marginal release protocols (BMS-POS dataset).

yields larger errors under LDP, this decreasing trend in SSE is a mathematical result of the expansion of the marginal table's domain size. On one hand, a larger domain introduces more value combinations to sum over, which would inflate the total SSE. On the other hand, since marginal tables are probability distributions whose frequencies sum to 1, this expanded domain forces both the final estimated frequency and target frequency of each value combination to approach zero, making the squared frequency differences between them significantly smaller. Ultimately, this shrinking effect dominates the growth of the value combinations, resulting in a smaller overall SSE.

Impact of N. Figure 7(c) shows the impact of the total number of users N. Note that in this experiment, we alter the number of genuine users and fake users simultaneously while keeping the fake user ratio unchanged. As N increases, the SSE of all evaluated methods decreases. This is because a larger number of users reduces the noise injected by the FO. Consequently, the aggregator obtains more accurate initial and final estimates, leading to a smaller overall SSE.

Impact of s_i. Figure 7(d) presents the impact of the attribute domain size s_i. As s_i increases, the SSE of attacks first decreases and then slowly increases. This trend is driven by two competing factors. Initially, similar to the impact of a larger k, a larger s_i expands the domain, forcing both the estimated and target frequencies to approach zero and reducing their squared differences. However, as s_i continues to grow, the extremely large domain causes severe data sparsity under the fixed user population. This amplifies LDP estimation noise, causing the SSE to rise.

Attacking Other LDP Protocols. As detailed in Sec. VI, we evaluate our attacks against other LDP marginal release protocols, including FC, AM, FT, and EM. While attacking the EM method requires a bi-level optimization framework, attacking the others only involves single-level optimization. To avoid ambiguity, we uniformly denote our methods as "Our Attack" in this section. We compare Our Attack with the Random, Input, and Ablation baselines. Note that the Ablation baseline is only applicable to the EM protocol, as EM is the only method that involves an iterative reconstruction workflow requiring bi-level optimization. Figure 8 presents the results. Compared to all baselines, Our Attack achieves the lowest SSE, highlighting its superior capability to manipulate final estimates towards the target marginal tables.

More Experimental Results. More experimental results on attack impact on the downstream tasks of marginal release, attack performance when there are multiple target attribute sets, and the efficiency and scalability of the attacks are provided in Appendix C.

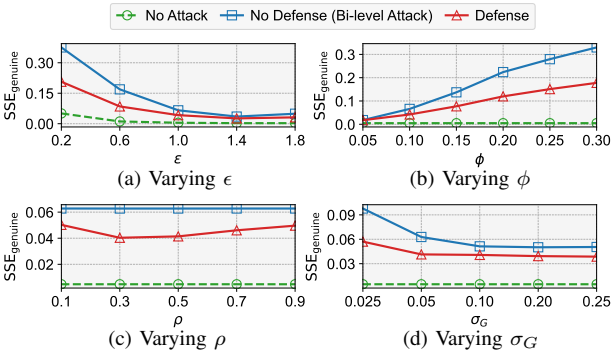
VIII. Possible Defense

Existing defenses mainly focus on mitigating untargeted/targeted attacks [46], [47], [48], [49], [50], [51], while defenses against fine-grained attacks only apply to mean/variance estimation [36] and streaming data publication [37]. Thus, in this section, we propose a defense against our attack. We design the defense based on the observation that to manipulate the final estimate towards an extreme target marginal table (e.g., in which a frequency is close to 1), the fake users must concentrate their reports on a specific value combination. This behavior introduces abnormal attribute correlations into the initial estimates. Our defense aims to mitigate the impact of attacks by suppressing such abnormal correlations. Specifically, our defense calibrates the initial estimates $\{\tilde{T}_{A_i}\}_{i=1}^{m}$ with the following steps:

Step 1: 1-D Marginalization. Assume the i-th view A_i contains l attributes $a_1, a_2, \ldots, a_l$. The aggregator first calculates the one-dimensional (1-D) marginal distribution for each attribute from the poisoned initial estimate $\tilde{T}_{A_i}$. Specifically, for any attribute $a_j \in A_i$ and its value $x \in V_{a_j}$, the 1-D frequency is $\tilde{T}_{\{a_j\}}(x) = \sum_{v \in V_{A_i}, v[a_j]=x} \tilde{T}_{A_i}(v)$, where $v[a_j]$ is the projection of the combination v onto the attribute a_j.

Step 2: Independent Approximation. Based on these 1-D marginals, the aggregator constructs an independent approximation for the view, denoted as $\tilde{T}^{\text{ind}}_{A_i}$. This approximation assumes that all l attributes in the view are independent. For any combination $v \in V_{A_i}$, this frequency is calculated as the product of its 1-D marginals, i.e., $\tilde{T}^{\text{ind}}_{A_i}(v) = \prod_{j=1}^{l} \tilde{T}_{\{a_j\}}(v[a_j])$. This step preserves the basic 1-D frequency distributions but removes the correlations among the attributes.

Step 3: Residual Calibration. For any combination $v \in V_{A_i}$, the aggregator then calculates the residual $R_i(v) = \tilde{T}_{A_i}(v) - \tilde{T}^{\text{ind}}_{A_i}(v)$, which represents the attribute correlations. For attacks that inject abnormal correlations, R_i exhibits large positive values at the specific value combinations. To mitigate this, the aggregator pulls the initial estimate towards the



(a) Varying ϵ (b) Varying ϕ (c) Varying ρ (d) Varying σ_G

Fig. 9: Defense performance w.r.t. different parameters (BMS-POS dataset with Gaussian target).

independent approximation using a parameter $\rho \in [0, 1]$ and derives the estimates $\tilde{T}^{\text{cal}}_{A_i}$ for all $v \in V_{A_i}$:

$$\tilde{T}^{\text{cal}}_{A_i}(v) = \tilde{T}_{A_i}(v) - \rho R_i(v) = (1-\rho)\tilde{T}_{A_i}(v) + \rho \tilde{T}^{\text{ind}}_{A_i}(v). \quad (19)$$

Finally, the calibrated estimates $\{\tilde{T}^{\text{cal}}_{A_i}\}_{i=1}^{m}$ replace the poisoned initial estimated marginal tables as the inputs for Phase 2 and Phase 3 of the CALM protocol to calculate the final estimate.

Experiments. We set $\rho = 0.5$ by default. We evaluate the defense using $SSE_{\text{genuine}} = \sum_{v \in V_A} (\tilde{T}_A(v) - M_A(v))^2$, measuring the distance between the final estimate $\tilde{T}_A$ and the genuine marginal table M_A. A smaller SSE_{genuine} indicates that the final estimate is closer to the genuine distribution, meaning a better defense. Figure 9 presents the results, where "No Attack" means executing the standard CALM protocol without any attack or defense. "No Defense" represents applying the Bi-level Attack without any defense. Overall, our defense achieves a smaller SSE_{genuine} than No Defense, demonstrating its effectiveness. In Figure 9(a), the SSE_{genuine} of defense decreases with the privacy budget ϵ since a larger ϵ reduces the noise introduced by LDP. In Figure 9(b), the SSE_{genuine} of defense increases with the fake user ratio ϕ, as injecting more fake users degrades the 1-D marginals in Step 1 and the independent approximation in Step 2. In Figure 9(c), the SSE_{genuine} of defense first decreases and then increases with ρ. This is because a smaller ρ makes the defense retain more malicious correlations, while an overly large ρ eliminates the entire correlations. Despite the general effectiveness of the defense, we find that its performance depends on the target marginal table. In Figure 9(d), we use Gaussian distributions with different standard deviations σ_G to form different Gaussian target tables. When σ_G is large (i.e., the target approaches a uniform distribution), the SSE_{genuine} of the defense is close to No Defense, indicating limited recovery. Conversely, when σ_G is small (i.e., the target approaches an extreme distribution with a frequency close to 1), the defense recovers the data significantly. This aligns with our design motivation: extreme targets force fake users to highly concentrate, creating prominent abnormal correlations that can be removed by the defense. Conversely, uniform targets scatter the fake reports, generating weak correlations that are difficult to isolate. How to recover the final estimate under arbitrary target marginal tables remains an important future work.

IX. Related Work

LDP Marginal Release. Wang *et al.* [10] proposed the *full contingency* (FC) method, which first estimates the joint distribution over the entire domain and then marginalizes it to obtain estimated marginals. Nguyen *et al.* [22] and Wang *et al.* [23] introduced the *all marginals* (AM) methods, which partition users into disjoint groups to estimate a predefined set of marginal tables. Cormode *et al.* [24] and Ren *et al.* [25] proposed the *fourier transformation* (FT) method and the *expectation maximization* (EM) method, respectively. The former reconstructs marginal tables by extracting frequency-domain Fourier coefficients, while the latter iteratively maximizes the likelihood of the perturbed user reports. Recently, Zhang *et al.* [26] introduced the CALM protocol. By adaptively selecting attribute views and enforcing consistency and maximum entropy processing, CALM achieves the SOTA data utility. Therefore, we select CALM as our primary attack objective, while our proposed attack framework naturally generalizes to all the aforementioned protocols.

Poisoning Attacks to LDP Protocols. Cheu *et al.* [27] initiated *untargeted poisoning* against LDP protocols for frequency estimation, manipulating the L_p norm distance pre- and post-attack. Tong *et al.* [28] proposed untargeted attacks to undermine the performance of LDP frequent itemset mining protocols. Li *et al.* [29] proposed such attacks to LDP protocols for numerical attributes. He *et al.* [30] proposed untargeted attacks to LDP-based Graph Neural Networks. Cao *et al.* [31] developed *targeted poisoning* to extremely promote certain items in LDP-based frequency estimation and heavy-hitter identification. Wu *et al.* [32] proposed attacks against LDP key-value data protocols. Zhan *et al.* [33] introduced targeted poisoning attacks for ranking estimation. He *et al.* [34] and Chowdhury *et al.* [35] attacked LDP graph metric estimation. Recently, Li *et al.* [36] pioneered *fine-grained poisoning* that can induce the estimated mean/variance towards the attacker-intended values. Li *et al.* [37] proposed fine-grained poisoning against LDP streaming data publication, while Li *et al.* [52] presented such poisoning against LDP range query. However, none of the existing attacks focus on fine-grained poisoning attacks against LDP marginal release.

X. Conclusion

We conduct a comprehensive study on fine-grained data poisoning attacks against LDP marginal release protocols. To attack the state-of-the-art CALM protocol, we propose a bi-level optimization framework. We model the aggregator's reconstruction workflow as the lower-level problem and formulate the attacker's objective as the upper-level problem. To efficiently solve this bi-level optimization problem, we utilize implicit differentiation to calculate the gradients and iteratively optimize the malicious reports. We also generalize our attack methodology to other mainstream LDP protocols. Extensive experiments demonstrate the effectiveness of the proposed attacks. Finally, we propose a defense mechanism to mitigate such threats. Our work highlights the security vulnerabilities of LDP marginal release protocols and calls for further research.

AI-Generated Content Acknowledgement

In this work, the authors utilized ChatGPT for code completion and correct grammatical errors in the manuscript. All AI-generated code completions and language suggestions were subsequently reviewed, modified as necessary, and approved by the authors, who take full responsibility for the entire content of the publication.

References

[1] J. C. Duchi, M. I. Jordan, and M. J. Wainwright, "Local privacy and statistical minimax rates," in *2013 IEEE 54th annual symposium on foundations of computer science*, 2013, pp. 429–438.

[2] J. Duchi, M. Jordan, and M. Wainwright, "Privacy aware learning," *Journal of the ACM (JACM)*, vol. 61, no. 6, pp. 1–57, 2014.

[3] S. Kasiviswanathan, H. Lee, N, S. Raskhodnikova, and A. Smith, "What can we learn privately?" *SIAM Journal on Computing*, vol. 40, no. 3, pp. 793–826, 2011.

[4] C. Dwork, "Differential privacy," in *International colloquium on automata, languages, and programming*, 2006, pp. 1–12.

[5] Ú. Erlingsson, V. Pihur, and A. Korolova, "Rappor: Randomized aggregatable privacy-preserving ordinal response," in *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*, 2014, pp. 1054–1067.

[6] Apple, "Learning with privacy at scale," 2017. [Online]. Available: https://machinelearning.apple.com/research/learning-with-privacy-at-scale

[7] B. Ding, J. Kulkarni, and S. Yekhanin, "Collecting telemetry data privately," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[8] H. Arcolezi, J. Couchot, B. Al Bouna, and X. Xiao, "Random sampling plus fake data: Multidimensional frequency estimates with local differential privacy," in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 2021, pp. 47–57.

[9] P. Kairouz, S. Oh, and P. Viswanath, "Extremal mechanisms for local differential privacy," *Journal of Machine Learning Research*, vol. 17, no. 17, pp. 1–51, 2016.

[10] T. Wang, J. Blocki, N. Li, and S. Jha, "Locally differentially private protocols for frequency estimation," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 729–745.

[11] L. Du, Z. Zhang, S. Bai, C. Liu, S. Ji, P. Cheng, and J. Chen, "Ahead: adaptive hierarchical decomposition for range query under local differential privacy," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1266–1288.

[12] N. Wang, Y. Wang, Z. Wang, J. Nie, Z. Wei, P. Tang, Y. Gu, and G. Yu, "Privnud: Effective range query processing under local differential privacy," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 2660–2672.

[13] L. Wang, Q. Ye, H. Hu, and X. Meng, "Pripl-tree: Accurate range query for arbitrary distribution under local differential privacy," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 3031–3044, 2024.

[14] J. Yang, T. Wang, N. Li, X. Cheng, and S. Su, "Answering multi-dimensional range queries under local differential privacy," *Proceedings of the VLDB Endowment*, vol. 14, no. 3, pp. 378–390, 2020.

[15] Y. Wang and X. Cheng, "Prism: Prefix-sum based range queries processing method under local differential privacy," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, 2022, pp. 433–445.

[16] Q. Ye, H. Hu, M. H. Au, X. Meng, and X. Xiao, "Lf-gdpr: A framework for estimating graph metrics with local differential privacy," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 10, pp. 4905–4920, 2020.

[17] J. Imola, T. Murakami, and K. Chaudhuri, "Locally differentially private analysis of graph statistics," in *30th USENIX security symposium (USENIX Security 21)*, 2021, pp. 983–1000.

[18] ——, "{Communication-Efficient} triangle counting under local differential privacy," in *31st USENIX security symposium (USENIX Security 22)*, 2022, pp. 537–554.

[19] Z. Qin, T. Yu, Y. Yang, I. Khalil, X. Xiao, and K. Ren, "Generating synthetic decentralized social graphs with local differential privacy," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 425–438.

[20] Y. Liu, T. Wang, Y. Liu, H. Chen, and C. Li, "Edge-protected triangle count estimation under relationship local differential privacy," *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[21] Y. He, K. Wang, W. Zhang, X. Lin, Y. Zhang, and W. Ni, "Robust privacy-preserving triangle counting under edge local differential privacy," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–26, 2025.

[22] T. Nguyên, X. Xiao, Y. Yang, S. Hui, H. Shin, and J. Shin, "Collecting and analyzing data from smart device users with local differential privacy," *arXiv preprint arXiv:1606.05053*, 2016.

[23] T. Wang, N. Li, and S. Jha, "Locally differentially private heavy hitter identification," *IEEE Trans. Dependable Secure Comput.*, 2019.

[24] G. Cormode, T. Kulkarni, and D. Srivastava, "Marginal release under local differential privacy," in *Proc. ACM SIGMOD*, 2018, pp. 131–146.

[25] X. Ren, C.-M. Yu, W. Yu, S. Yang, X. Yang, J. McCann, and S. Philip, "LoPub: High-dimensional crowdsourced data publication with local differential privacy," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 9, pp. 2151–2166, 2018.

[26] Z. Zhang, T. Wang, N. Li, S. He, and J. Chen, "Calm: Consistent adaptive local marginal for marginal release under local differential privacy," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 212–229.

[27] A. Cheu, A. Smith, and J. Ullman, "Manipulation attacks in local differential privacy," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 883–900.

[28] W. Tong, H. Chen, J. Niu, and S. Zhong, "Data poisoning attacks to locally differentially private frequent itemset mining protocols," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024.

[29] X. Li, Z. Li, N. Li, and W. Sun, "On the robustness of ldp protocols for numerical attributes under data poisoning attacks," *32th Annual Network And Distributed System Security Symposium (NDSS 2025)*, 2025.

[30] L. He, C. Li, P. Tang, L. Zhang, and S. Su, "Devil's hand: Data poisoning attacks to locally private graph learning protocols," *arXiv preprint arXiv:2506.09803*, 2025.

[31] X. Cao, J. Jia, and N. Z. Gong, "Data poisoning attacks to local differential privacy protocols," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 947–964.

[32] Y. Wu, X. Cao, J. Jia, and N. Z. Gong, "Poisoning attacks to local differential privacy protocols for {Key-Value} data," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 519–536.

[33] P. Zhan, P. Tang, Y. Li, P. Wei, and S. Guo, "Poisoning attacks to local differential privacy for ranking estimation," in *Proceedings of the 2025 on ACM SIGSAC Conference on Computer and Communications Security*, 2025.

[34] X. He, K. Huang, Q. Ye, and H. Hu, "Data poisoning attacks to local differential privacy protocols for graphs," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024.

[35] A. Roy Chowdhury, J. Imola, and K. Chaudhuri, "Robust locally differentially private graph analysis," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, 2025, pp. 635–650.

[36] X. Li, N. Li, W. Sun, N. Z. Gong, and H. Li, "Fine-grained poisoning attack to local differential privacy protocols for mean and variance estimation," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 1739–1756.

[37] X. Li, X. Ren, S. Yang, L. Shi, C.-M. Yu, and Q. Han, "Fine-grained manipulation attacks to local differential privacy protocols for data streams," *IEEE Transactions on Knowledge and Data Engineering*, 2026.

[38] K. Thomas, D. McCoy, C. Grier, A. Kolcz, and V. Paxson, "{Trafficking} fraudulent accounts: The role of the underground market in twitter spam and abuse," in *22nd USENIX Security Symposium (USENIX Security 13)*, 2013, pp. 195–210.

[39] L. H. X. Ng and K. M. Carley, "A global comparison of social media bot and human characteristics," *Scientific Reports*, vol. 15, no. 1, p. 10973, 2025.

[40] "General data protection regulation," 2016. [Online]. Available: https://gdpr-info.eu/

[41] "California consumer privacy act," 2024. [Online]. Available: https://oag.ca.gov/privacy/ccpa

[42] Z. Zheng, R. Kohavi, and L. Mason, "Real world performance of association rule algorithms," in *Proceedings of the seventh ACM SIGKDD*

international conference on Knowledge discovery and data mining, 2001, pp. 401–406.
[43] "Kosarak." [Online]. Available: https://fimi.ua.ac.be/data
[44] "Adult." [Online]. Available: https://archive.ics.uci.edu/dataset/2/adult
[45] "Bfive." [Online]. Available: https://www.kaggle.com/datasets/tunguz/big-five-personality-test
[46] X. Sun, Q. Ye, H. Hu, J. Duan, T. Wo, J. Xu, and R. Yang, "Ldprecover: Recovering frequencies from poisoning attacks against local differential privacy," in *2024 IEEE 40th international conference on data engineering (ICDE)*, 2024.
[47] K. Huang, G. Ouyang, Q. Ye, H. Hu, B. Zheng, X. Zhao, R. Zhang, and X. Zhou, "Ldpguard: Defenses against data poisoning attacks to local differential privacy protocols," *IEEE Transactions on Knowledge and Data Engineering*, 2024.
[48] P. Zhao, Z. Zhang, J. Dong, J. Wu, S. Wang, Z. Liu, and Y. Gao, "An attack-agnostic defense framework against manipulation attacks under local differential privacy," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025.
[49] S. Wang, Q. Luo, Y. Hu, L. Wang, Q. Xu, C. Yang, Z. Qin, K. Ren, and W. Dong, "Defense against poisoning attacks under shuffle-dp," *Proceedings of the ACM on Management of Data*, vol. 4, no. 1 (SIGMOD), pp. 1–27, 2026.
[50] W. Dong, Y. Cao, C. Wang, and W.-B. Lee, "Doppio: Communication-efficient and secure multi-party shuffle differential privacy," *Proceedings of the VLDB Endowment*, vol. 19, no. 2, pp. 113–126, 2025.
[51] T. Murakami, Y. Sei, and R. Eriguchi, "Augmented shuffle protocols for accurate and robust frequency estimation under differential privacy," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 3892–3911.
[52] X. Li, W. Chen, and X. Ren, "Fine-grained manipulation attacks to local differential privacy protocols for range query," in *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. IEEE, 2026, pp. 1279–1291.

APPENDIX

A. Proofs of Theorems

1) Proof of Theorem 1

Denote the collection of all fake count vectors as $\tilde{\mathbf{c}}^{(t)} = \{\tilde{c}_i^{(t)}\}_{i=1}^m$. Assuming the optimal solution $\tilde{\mathbf{T}}_t^*$ lies in the relative interior of the probability simplices, the non-negativity constraints are strictly satisfied. Thus, the Lagrangian of the lower-level problem only needs to incorporate the sum-to-one equality constraints, which can be formulated as

$$\mathcal{J}\left(\tilde{\mathbf{T}}_t^*, \kappa_t; \tilde{\mathbf{c}}^{(t)}\right) = \mathcal{J}_{\text{low}} + \kappa_{A,t}\left(\mathbf{1}_{|V_A|}^\top \tilde{T}_{A,t}^* - 1\right) + \sum_{i=1}^m \kappa_{i,t}\left(\mathbf{1}_{|V_{A_i}|}^\top \tilde{T}_{A_i,t}^{c,*} - 1\right). \quad (20)$$

where $\kappa_t = (\kappa_{A,t}, \kappa_{1,t}, \ldots, \kappa_{m,t})$ are the Lagrangian multipliers, and $\mathbf{1}$ denotes the all-one vectors. Since $\tilde{\mathbf{T}}_t^*$ is optimal, it satisfies the Karush–Kuhn–Tucker (KKT) conditions. This implies that the gradient of the Lagrangian with respect to the primal variables is zero and the equality constraints are perfectly met. Specifically, we can express these conditions as a combined function F that equals zero:

$$F\left(\tilde{\mathbf{T}}_t^*, \kappa_t; \tilde{\mathbf{c}}^{(t)}\right) = \begin{bmatrix} \nabla_{\tilde{\mathbf{T}}} \mathcal{J}\left(\tilde{\mathbf{T}}_t^*, \kappa_t; \tilde{\mathbf{c}}^{(t)}\right) \\ \mathbf{1}_{|V_A|}^\top \tilde{T}_{A,t}^* - 1 \\ \mathbf{1}_{|V_{A_1}|}^\top \tilde{T}_{A_1,t}^{c,*} - 1 \\ \vdots \\ \mathbf{1}_{|V_{A_m}|}^\top \tilde{T}_{A_m,t}^{c,*} - 1 \end{bmatrix} = \mathbf{0}. \quad (21)$$

Differentiating the KKT system $F(\tilde{\mathbf{T}}_t^*, \kappa_t; \tilde{\mathbf{c}}^{(t)}) = 0$ with respect to $\tilde{c}_i^{(t)}$, we have $\frac{\partial F}{\partial(\tilde{\mathbf{T}}_t^*, \kappa_t)} \frac{\partial(\tilde{\mathbf{T}}_t^*, \kappa_t)}{\partial \tilde{c}_i^{(t)}} + \frac{\partial F}{\partial \tilde{c}_i^{(t)}} = 0$, which implies $\frac{\partial(\tilde{\mathbf{T}}_t^*, \kappa_t)}{\partial \tilde{c}_i^{(t)}} = -\left(\frac{\partial F}{\partial(\tilde{\mathbf{T}}_t^*, \kappa_t)}\right)^{-1} \frac{\partial F}{\partial \tilde{c}_i^{(t)}}$. With the chain rule, the gradient $\nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)}$ is

$$\begin{aligned} \nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)} &= \left(\frac{\partial(\tilde{\mathbf{T}}_t^*, \kappa_t)}{\partial \tilde{c}_i^{(t)}}\right)^\top \nabla_{(\tilde{\mathbf{T}}_t^*, \kappa_t)} \mathcal{L}_{\text{atk}}^{(t)} \\ &= -\left(\frac{\partial F}{\partial \tilde{c}_i^{(t)}}\right)^\top \left(\frac{\partial F}{\partial(\tilde{\mathbf{T}}_t^*, \kappa_t)}\right)^{-\top} \nabla_{(\tilde{\mathbf{T}}_t^*, \kappa_t)} \mathcal{L}_{\text{atk}}^{(t)}. \end{aligned} \quad (22)$$

Directly calculating the inverse of the Jacobian $\partial F / \partial(\tilde{\mathbf{T}}_t^*, \kappa_t)$ is computationally expensive. Therefore, we introduce the adjoint variable ξ defined by solving the following linear system:

$$\left(\frac{\partial F}{\partial(\tilde{\mathbf{T}}_t^*, \kappa_t)}\right)^\top \xi = \nabla_{(\tilde{\mathbf{T}}_t^*, \kappa_t)} \mathcal{L}_{atk}^{(t)}, \quad (23)$$

where $\nabla_{(\tilde{\mathbf{T}}_t^*, \kappa_t)} \mathcal{L}_{atk}^{(t)} = [(\tilde{T}_{A,t}^* - T_A^{tar})^\top, \mathbf{0}_{\sum_{i=1}^m |V_{A_i}| + m + 1}^\top]^\top$ and $\mathbf{0}_D$ is an all-zero vector of dimension D. With ξ, the gradient can be written as $\nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)} = -(\frac{\partial F}{\partial \tilde{c}_i^{(t)}})^\top \xi$.It remains to calculate $\partial F / \partial \tilde{c}_i^{(t)}$. The fake count vector $\tilde{c}_i^{(t)}$ affects the KKT system only through the initial estimated marginal table $\tilde{T}_{A_i}^{(t)}$. Thus, we have $\frac{\partial F}{\partial \tilde{c}_i^{(t)}} = \frac{\partial F}{\partial \tilde{T}_{A_i}^{(t)}} \frac{\partial \tilde{T}_{A_i}^{(t)}}{\partial \tilde{c}_i^{(t)}}$. Differentiating Eq. (7), we have $\frac{\partial \tilde{T}_{A_i}^{(t)}}{\partial \tilde{c}_i^{(t)}} = \frac{1}{(n_i^e + \tilde{n}_i)(p_i - q_i)} I_{|V_{A_i}|}$, where $I_{|V_{A_i}|}$ is a $|V_{A_i}| \times |V_{A_i}|$ identity matrix.Moreover, $\tilde{T}_{A_i}^{(t)}$ only appears in the term $\alpha_i \|\tilde{T}_{A_i,t}^{c,*} - \tilde{T}_{A_i}^{(t)}\|_2^2$ of $\mathcal{J}_{\text{low}}$ in $\mathcal{J}$. Thus,

$$\frac{\partial F}{\partial \tilde{T}_{A_i}^{(t)}} = \begin{bmatrix} \mathbf{0}_{(|V_A| + \sum_{j=1}^{i-1} |V_{A_j}|) \times |V_{A_i}|} \\ -2\alpha_i I_{|V_{A_i}|} \\ \mathbf{0}_{(\sum_{j=i+1}^m |V_{A_j}| + m + 1) \times |V_{A_i}|} \end{bmatrix}, \quad (24)$$

where $\mathbf{0}_{D_1 \times D_2}$ is an all-zero matrix of dimension $D_1 \times D_2$.Substituting the above derivatives back to Eq. (22) gives

$$\nabla_{\tilde{c}_i^{(t)}} \mathcal{L}_{\text{atk}}^{(t)} = -\frac{1}{(n_i^e + \tilde{n}_i)(p_i - q_i)} \left(\frac{\partial F}{\partial \tilde{T}_{A_i}^{(t)}}\right)^\top \xi. \quad (25)$$

2) Proof of Theorem 2

Fix any feasible fake count vectors $\{\tilde{\mathbf{c}}_i\}_{i=1}^m$, and let $\{\tilde{T}_{A_i}\}_{i=1}^m$ be the resulting initial marginal tables. Let $(\tilde{T}_A, \{\tilde{T}_{A_i}^c\}_{i=1}^m)$ be an optimal solution to the lower-level reconstruction problem in Eq. (9) with these tables as inputs. Since $\lambda > 0$, the optimal reconstructed marginal table lies in the interior of Δ_A, i.e., $\tilde{T}_A(v) > 0$ for all $v \in V_A$. Let $\mathcal{J}_{\text{low}}$ denote the lower-level objective. Only the entropy term and the reconstruction consistency penalties depend on $\tilde{T}_A$. Differentiating with respect to $\tilde{T}_A$, while holding $\{\tilde{T}_{A_i}^c\}_{i=1}^m$ fixed, gives

$$\nabla_{\tilde{T}_A} \mathcal{J}_{\text{low}} = \lambda(\log \tilde{T}_A + \mathbf{1}) + 2\sum_{i=1}^m \gamma_i G_i^\top (G_i \tilde{T}_A - H_i \tilde{T}_{A_i}^c). \quad (26)$$

At this interior optimum, the first-order stationarity condition under the normalization constraint $\mathbf{1}^\top \tilde{T}_A = 1$ is

$$\lambda(\log \tilde{T}_A + \mathbf{1}) + 2\sum_{i=1}^m \gamma_i G_i^\top (G_i \tilde{T}_A - H_i \tilde{T}_{A_i}^c) + \nu \mathbf{1} = 0, \quad (27)$$

where ν is the Lagrange multiplier associated with the normalization constraint. For any $u \in \mathcal{N}$, we have $\mathbf{1}^\top u = 0$ and $G_i u = 0$ for all i. Left-multiplying the stationarity condition by $u^\top$ yields

$$0 = \lambda u^\top (\log \tilde{T}_A + \mathbf{1}) + 2\sum_{i=1}^m \gamma_i (G_i u)^\top (G_i \tilde{T}_A - H_i \tilde{T}_{A_i}^c) + \nu u^\top \mathbf{1}. \quad (28)$$

Since $G_i u = 0$ and $u^\top \mathbf{1} = 0$, all terms except $\lambda u^\top \log \tilde{T}_A$ vanish. As $\lambda > 0$, it follows that $u^\top \log \tilde{T}_A = 0$ for every $u \in \mathcal{N}$. Combining the definition of r with the Cauchy–Schwarz inequality yields,

$$\begin{aligned} r &= \max_{\substack{u \in \mathcal{N} \\ \|u\|_2 \le 1}} |u^\top \log T_A^{\text{tar}}| \\ &= \max_{\substack{u \in \mathcal{N} \\ \|u\|_2 \le 1}} |u^\top (\log T_A^{\text{tar}} - \log \tilde{T}_A)| \le \|\log T_A^{\text{tar}} - \log \tilde{T}_A\|_2. \end{aligned} \quad (29)$$

Let $\delta = \|\tilde{T}_A - T_A^{\text{tar}}\|_2$. Suppose first that $\delta < \min_{v \in V_A} T_A^{\text{tar}}(v)$. For every $v \in V_A$,

$$\tilde{T}_A(v) \ge T_A^{\text{tar}}(v) - |\tilde{T}_A(v) - T_A^{\text{tar}}(v)| \ge \min_{v \in V_A} T_A^{\text{tar}}(v) - \delta > 0. \quad (30)$$

Applying the mean value theorem to the logarithm componentwise yields,

$$|\log T_A^{\text{tar}}(v) - \log \tilde{T}_A(v)| \le \frac{|T_A^{\text{tar}}(v) - \tilde{T}_A(v)|}{\min_{v \in V_A} T_A^{\text{tar}}(v) - \delta}. \quad (31)$$

Consequently,

$$r \le \|\log T_A^{\mathrm{tar}} - \log \tilde{T}_A\|_2 \le \frac{\|\tilde{T}_A - T_A^{\mathrm{tar}}\|_2}{\min_{v\in V_A} T_A^{\mathrm{tar}}(v) - \delta} = \frac{\delta}{\min_{v\in V_A} T_A^{\mathrm{tar}}(v) - \delta}. \quad (32)$$

Rearranging yields,

$$\delta \ge \left(\min_{v\in V_A} T_A^{\mathrm{tar}}(v)\right) \frac{r}{1+r}. \quad (33)$$

If instead $\delta \ge \min_{v\in V_A} T_A^{\mathrm{tar}}(v)$, the same bound follows immediately from $0 \le r/(1+r) < 1$. Thus, the bound holds in both cases. Since $\mathcal{L}_{\mathrm{atk}} = \frac{1}{2}\|\tilde{T}_A - T_A^{\mathrm{tar}}\|_2^2 = \frac{1}{2}\delta^2$, the attack loss satisfies

$$\mathcal{L}_{\mathrm{atk}} \ge \frac{1}{2}\left(\min_{v\in V_A} T_A^{\mathrm{tar}}(v)\right)^2 \left(\frac{r}{1+r}\right)^2. \quad (34)$$

B. Attacking Other LDP Protocols

The attack methodology proposed against CALM can be naturally extended to other LDP marginal release protocols by targeting their specific estimates.

1) Attacking the FC Method

The *full contingency* (FC) method [10] first estimates the full joint frequency table over the entire domain $D = V_1 \times V_2 \times \cdots \times V_d$. Without attack, let the joint frequency table be $F = (F(x))_{x\in D}$. The aggregator estimates the final estimated marginal table T_A by directly marginalizing F over the attribute set A:

$$T_A(v) = \sum_{\substack{x\in D \\ x[A]=v}} F(x), \quad v \in V_A. \quad (35)$$

where $x[A] = v$ indicates that the projection of the combination x onto the target attribute set A equals v. Under attack, the attacker generates the fake count vector $\tilde{c}_{\mathrm{FC}} = (\tilde{c}_{\mathrm{FC}}(x))_{x\in D}$ over the entire domain D to distort the joint frequency table, yielding the poisoned joint table $\tilde{F}(x; \tilde{c}_{\mathrm{FC}})$. Consequently, the poisoned final estimated marginal table $\tilde{T}_A$ is obtained as

$$\tilde{T}_A(v) = \sum_{\substack{x\in D \\ x[A]=v}} \tilde{F}(x; \tilde{c}_{\mathrm{FC}}), \quad v \in V_A. \quad (36)$$

Since the mapping from the joint table $\tilde{F}$ to the final estimate $\tilde{T}_A$ is a simple linear summation without any iterative reconstruction, the attack can be formulated as a straightforward single-level optimization problem:

$$\min_{\tilde{c}_{\mathrm{FC}}\in\mathcal{C}_{\mathrm{FC}}} \sum_{v\in V_A} \left(\sum_{\substack{x\in D \\ x[A]=v}} \tilde{F}(x; \tilde{c}_{\mathrm{FC}}) - T_A^{\mathrm{tar}}(v)\right)^2, \quad (37)$$

where $\mathcal{C}_{\mathrm{FC}}$ is the feasible region determined by the local perturbation mechanism.

2) Attacking the AM Method

The *all marginals* (AM) method [22], [23] avoids estimating the full joint table by directly estimating a predefined set of marginal tables $\{T_{A_1}, \ldots, T_{A_s}\}$ by partitioning users into disjoint groups, each reporting on a fixed attribute set A_i. Let $\mathcal{I}_A = \{i \mid A \subseteq A_i\}$ denote the indices of the fixed attribute sets that cover the target attribute set A. Without attack, the aggregator constructs the final estimated marginal table T_A by directly marginalizing any covering table T_{A_i} ($i \in \mathcal{I}_A$):

$$T_A(v) = \sum_{\substack{u\in V_{A_i} \\ u[A]=v}} T_{A_i}(u), \quad v \in V_A. \quad (38)$$

where $u[A] = v$ indicates that the projection of the combination u onto the target attribute set A equals v. Under attack, the attacker manipulates the fake count vectors $\{\tilde{c}_i\}_{i\in\mathcal{I}_A}$ to distort the corresponding marginal tables, yielding the poisoned tables $\tilde{T}_{A_i}$. Consequently, the poisoned final estimated marginal table $\tilde{T}_A$ is obtained as

$$\tilde{T}_A(v) = \sum_{\substack{u\in V_{A_i} \\ u[A]=v}} \tilde{T}_{A_i}(u; \tilde{c}_i), \quad v \in V_A. \quad (39)$$

Since this mapping is a direct linear summation without any iterative reconstruction, the attack is formulated as a single-level optimization problem:

$$\min_{\{\tilde{c}_i\}_{i\in\mathcal{I}_A}\in\mathcal{C}_{\mathrm{AM}}} \sum_{v\in V_A} \left(\sum_{\substack{u\in V_{A_i} \\ u[A]=v}} \tilde{T}_{A_i}(u; \tilde{c}_i) - T_A^{\mathrm{tar}}(v)\right)^2, \quad (40)$$

where $\mathcal{C}_{\mathrm{AM}}$ is the corresponding feasible region.

3) Attacking the FT Method

The *fourier transformation* (FT) method [24] estimates the relevant Fourier coefficients of the data distribution instead of the marginal tables directly. For binary attributes, let $\Gamma_A = \{\alpha \in \{0,1\}^d \mid \alpha_{\bar{A}} = \mathbf{0}\}$ be the set of indices for Fourier coefficients relevant to the target attribute set A, where $\bar{A} = \mathcal{A} \setminus A$. Without attack, the aggregator estimates the Fourier coefficients $\theta_{b(\alpha)}$ and reconstructs the final estimated marginal table T_A via the inverse Fourier transform:

$$T_A(v) = \sum_{\alpha\in\Gamma_A} \theta_{b(\alpha)} \left(\sum_{\substack{\eta\in D \\ \eta[A]=v}} \omega_{b(\alpha),b(\eta)}\right), \quad v \in V_A. \quad (41)$$

where ω represents the elements of the Fourier basis matrix. Under attack, the attacker controls the fake count vector $\tilde{c}_{\mathrm{FT}} = (\tilde{c}_{b(\alpha)})_{\alpha\in\Gamma_A}$ to distort the recovered Fourier coefficients to $\tilde{\theta}_{b(\alpha)}(\tilde{c}_{\mathrm{FT}})$. The final estimated marginal table $\tilde{T}_A$ after attack is calculated as

$$\tilde{T}_A(v) = \sum_{\alpha\in\Gamma_A} \tilde{\theta}_{b(\alpha)}(\tilde{c}_{\mathrm{FT}}) \left(\sum_{\substack{\eta\in D \\ \eta[A]=v}} \omega_{b(\alpha),b(\eta)}\right). \quad (42)$$

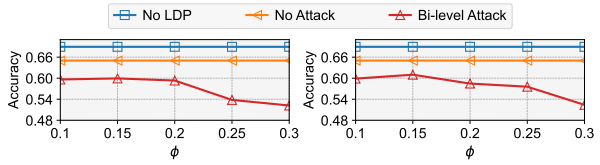
Because $\tilde{T}_A$ is explicitly calculated from the Fourier coefficients without any iterative reconstruction, the attack is modeled as a single-level optimization problem:

$$\min_{\tilde{c}_{\mathrm{FT}}\in\mathcal{C}_{\mathrm{FT}}} \sum_{v\in V_A} \left(\sum_{\alpha\in\Gamma_A} \tilde{\theta}_{b(\alpha)}(\tilde{c}_{\mathrm{FT}}) \cdot \left(\sum_{\substack{\eta\in D \\ \eta[A]=v}} \omega_{b(\alpha),b(\eta)}\right) - T_A^{\mathrm{tar}}(v)\right)^2 \quad (43)$$

where $\mathcal{C}_{\mathrm{FT}}$ is the feasible region determined by the FT protocol constraints.

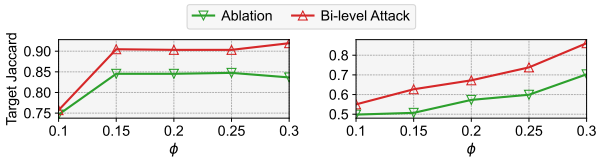
4) Attacking the EM Method

The *expectation maximization* (EM) method [25] estimates marginal tables by running the EM algorithm to maximize the likelihood of the observed independent perturbed reports.



(a) Uniform target (b) Gaussian target

Fig. 10: Impact of the attack on downstream data synthesis tasks, varying ϕ (Kosarak dataset).



(a) Exponential target (b) Bimodal target

Fig. 11: Impact on association rule mining, varying ϕ (BMS-POS dataset, Exponential (left) and Bimodal (right) targets).

Let $\mathcal{Y}_A$ be the domain of the perturbed reports on the target attribute set A. Without attack, the aggregator utilizes $C_A(w)$ (i.e., the fraction of users whose perturbed report on the attribute set A is $w \in \mathcal{Y}_A$) to reconstruct the final estimated marginal table T_A by solving

$$T_A \in \arg\max_{T \in \Delta_A} \sum_{w \in \mathcal{Y}_A} C_A(w) \log \left(\sum_{v \in V_A} T(v) \Pr[w \mid v] \right). \quad (44)$$

where Δ_A is the probability simplex for the target table. Under attack, the attacker introduces the fake count vector $\tilde{c}_{\text{EM}} = (\tilde{c}_{\text{EM}}(w))_{w \in \mathcal{Y}_A}$, altering the estimates from $C_A(w)$ to $\tilde{C}_A(w; \tilde{c}_{\text{EM}})$. The final estimate $\tilde{T}_A$ after attack is then iteratively solved as

$$\tilde{T}_A \in \arg\max_{T \in \Delta_A} \sum_{w \in \mathcal{Y}_A} \tilde{C}_A(w; \tilde{c}_{\text{EM}}) \cdot \log \left(\sum_{v \in V_A} T(v) \Pr[w \mid v] \right) \quad (45)$$

Since this mapping involves an iterative likelihood maximization process, the attack is formulated as a bi-level optimization problem:

$$\min_{\tilde{c}_{\text{EM}} \in \mathcal{C}_{\text{EM}}} \quad \sum_{v \in V_A} \left(\tilde{T}_A(v) - T_A^{\text{tar}}(v) \right)^2 \quad (46)$$

$$\text{s.t.} \quad \tilde{T}_A \in \arg\max_{T \in \Delta_A} \sum_{w \in \mathcal{Y}_A} \tilde{C}_A(w; \tilde{c}_{\text{EM}}) \cdot \log \left(\sum_{v \in V_A} T(v) \Pr[w \mid v] \right)$$

where $\mathcal{C}_{\text{EM}}$ is the corresponding feasible region.

5) Crafting Malicious Reports.

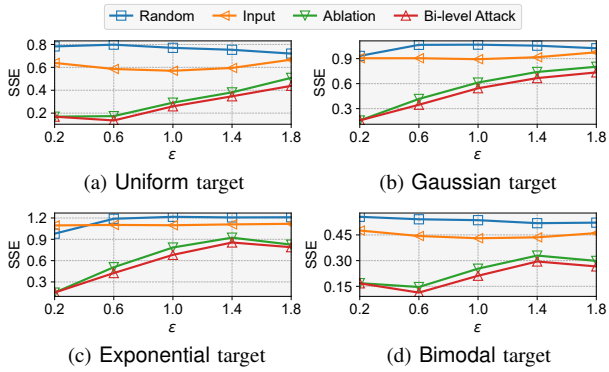
Once the respective optimization problem (single-level for FC, AM and FT, or bi-level for EM) is solved, the attacker obtains the optimal continuous fake count vector (i.e., $\tilde{c}_i^*$, $\tilde{c}_{\text{FT}}^*$, or $\tilde{c}_{\text{EM}}^*$). Similar to Sec. V-D, the attacker rounds these optimal continuous variables into valid integer counts and constructs discrete malicious reports, thus completing the attack.

C. More Experimental Results

1) Validation on Downstream Tasks.

We assess the practical impact of our Bi-level Attack on two downstream tasks: data synthesis and association rule mining.

Data Synthesis. To demonstrate the practical consequences of our Bi-level Attack, we evaluate its impact on downstream data synthesis tasks. We use 80% of the records in the dataset to estimate the marginal tables, from which a synthetic dataset [26] is generated and then used to train a Decision Tree classifier. During the LDP estimation, the attacker injects malicious reports to manipulate the marginal estimates, degrading the utility of the synthesized data. We use the remaining 20% of the records as the testing set to evaluate the classifier's accuracy. Figure 10 shows the results. Here, "No LDP" refers to training directly on the raw data, while "No Attack" represents the standard LDP release without any data poisoning. As shown, our attack can significantly reduce the classification accuracy. Also, a larger fake user ratio ϕ enhances the attack's impact on downstream tasks.



(a) Uniform target (b) Gaussian target

(c) Exponential target (d) Bimodal target

Fig. 12: Attacking multiple target attribute sets, varying ϵ (BMS-POS dataset).

Association Rule Mining. We evaluate the impact of our attack on association rule mining, focusing on the relatively small SSE differences between Ablation and Bi-level Attack. We use Target Jaccard similarity to compare the association rules mined from the poisoned final marginal table with those from the target marginal table. Let $\mathcal{R}_p$ and $\mathcal{R}_t$ denote these two rule sets, respectively. The similarity is defined as $J_{\text{tar}} = \frac{|\mathcal{R}_p \cap \mathcal{R}_t|}{|\mathcal{R}_p \cup \mathcal{R}_t|}$, where a higher value indicates greater overlap with the target rules. Figure 11 shows that Bi-level Attack achieves approximately 13.6% higher Target Jaccard similarity than Ablation, despite their small SSE differences. This indicates that even small differences in SSE can lead to more noticeable changes in downstream association rules.

2) Attacking Multiple Target Attribute Sets.

We further extend our Bi-level Attack to jointly manipulate multiple target marginal tables. Specifically, let $\mathcal{A}_{\text{tar}} = \{A^{(1)}, A^{(2)}, \ldots, A^{(R)}\}$ denote R target attribute sets, each associated with a predefined target marginal table $T_{A^{(r)}}^{\text{tar}}$. The attacker aims to simultaneously manipulate these targets using the same collection of fake reports. Accordingly, we extend the upper-level attack objective to the average SSE over all targets: $\mathcal{L}_{\text{joint}} = \frac{1}{2R} \sum_{r=1}^{R} \left\| \tilde{T}_{A^{(r)}} - T_{A^{(r)}}^{\text{tar}} \right\|_2^2$. The lower-level reconstruction for each target follows the same formulation as Eq. (10), using the shared poisoned initial marginal tables. The implicit gradients of individual targets are averaged to update the same fake count vectors, while the original protocol feasibility constraints remain unchanged. Figure 12 shows the joint attack performance under varying ϵ. As ϵ increases, the SSE generally rises, since a larger $p_i - q_i$ reduces the influence

of malicious reports on the initial view estimates, making it harder to shift the final marginals toward the targets. Bi-level Attack maintains the lowest SSE across all four settings.

3) Efficiency and Scalability.

Table II presents the impact of d, k, N, and s_i on the computational cost of Bi-level Attack. Increasing d introduces more views and consistency constraints, with a slight increase in peak memory usage. As k increases, CALM reconstructs a larger target marginal table, slightly increasing runtime while leaving peak memory nearly unchanged. Increasing N raises runtime, whereas memory grows gradually. Finally, increasing s_i enlarges both the view marginal tables and the final estimated marginal table, increasing runtime and peak memory usage.

TABLE II: Computational cost of the bi-level attack, measuring runtime (s) and peak memory (MB).

d	Runtime/Memory	k	Runtime/Memory	N	Runtime/Memory	s_i	Runtime/Memory
4	5.6/187.2	3	32.3/188.1	2^{14}	13.9/184.9	2	33.4/189.9
8	34.4/188.7	4	33.3/187.9	2^{15}	33.0/186.5	3	35.5/193.2
12	99.9/194.4	5	34.2/188.0	2^{16}	33.2/188.3	4	39.8/202.3
16	79.8/195.6	6	34.8/188.4	2^{17}	33.3/194.5	5	48.7/234.0
20	68.4/197.4	7	36.5/189.3	2^{18}	167.3/212.1	6	85.3/318.1